

Operating Systems

Ch 8: Deadlocks

Dr. 何明昕 HE Mingxin, MX, Max

Send your email to c.max@yeah.net with
a subject like: *OS-id-Andy: On What ...*

Download from c.program@yeah.net

[/文件中心/网盘/OperatingSystems26S](#)

Book site: <https://codex.cs.yale.edu/avi/os-book/OS10/index.html>





Outline

- System Model
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock





Exercises 8.21, 8.27, 8.28 (By hands only)

8.21 Consider the following snapshot of a system:

	<u>Allocation</u>	<u>Max</u>
	<i>A B C D</i>	<i>A B C D</i>
T_0	2 1 0 6	6 3 2 7
T_1	3 3 1 3	5 4 1 5
T_2	2 3 1 2	6 6 1 4
T_3	1 2 3 4	4 3 4 5
T_4	3 0 3 0	7 2 6 1

What are the contents of the *Need* matrix?





Exercises 8.21, 8.27, 8.28

8.27 Consider the following snapshot of a system:

	<u>Allocation</u>	<u>Max</u>
	<i>A B C D</i>	<i>A B C D</i>
T_0	1 2 0 2	4 3 1 6
T_1	0 1 1 2	2 4 2 4
T_2	1 2 4 0	3 6 5 1
T_3	1 2 0 1	2 6 2 3
T_4	1 0 0 1	3 1 1 2

Using the banker's algorithm, determine whether or not each of the following states is unsafe. If the state is safe, illustrate the order in which the threads may complete. Otherwise, illustrate why the state is unsafe.

- a. *Available* = (2, 2, 2, 3)
- b. *Available* = (4, 4, 1, 1)
- c. *Available* = (3, 0, 1, 4)
- d. *Available* = (1, 5, 2, 2)





Exercises 8.21, 8.27, 8.28

8.28 Consider the following snapshot of a system:

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C D	A B C D	A B C D
T_0	3 1 4 1	6 4 7 3	2 2 2 4
T_1	2 1 0 2	4 2 3 2	
T_2	2 4 1 3	2 5 3 3	
T_3	4 1 1 0	6 3 3 2	
T_4	2 2 2 1	5 6 7 5	

Answer the following questions using the banker's algorithm:

- Illustrate that the system is in a safe state by demonstrating an order in which the threads may complete.
- If a request from thread T_4 arrives for (2, 2, 2, 4), can the request be granted immediately?
- If a request from thread T_2 arrives for (0, 1, 1, 0), can the request be granted immediately?
- If a request from thread T_3 arrives for (2, 2, 1, 2), can the request be granted immediately?

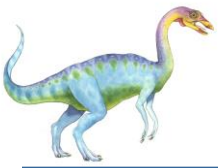




Chapter Objectives

- Illustrate how deadlock can occur when mutex locks are used
- Define the four necessary conditions that characterize deadlock
- Identify a deadlock situation in a resource allocation graph
- Evaluate the four different approaches for preventing deadlocks
- Apply the banker's algorithm for deadlock avoidance
- Apply the deadlock detection algorithm
- Evaluate approaches for recovering from deadlock





In a multiprogramming environment, several processes may compete for a finite number of resources.

A process requests resources; if the resources are not available at that time, the process enters a **waiting** state.

Sometimes, a waiting process is **never** again able to change state, because the resources it has requested are held by other **waiting** processes.

This situation is called a **deadlock**.

In this chapter, we describe methods that an operating system can use to prevent or deal with deadlocks





An Example

- ❑ When two trains approach each other at a crossing, both shall come to a full stop, and neither shall start up again until the other has gone
- ❑ A law passed by the Kansas legislature early in the 20th century





System Model

- ❑ System consists of a finite number of resources
- ❑ A number of competing processes
- ❑ Partitioned into several **resource types** $R_1, R_2, \dots, R_m$
 - *CPU cycles, memory space, I/O devices*
- ❑ Each resource type R_i has W_i instances.
- ❑ Synchronization tools, such as mutex locks and semaphores are also considered system resources
 - A common source of deadlock
- ❑ Each process utilizes a resource in only the sequence:
 - **request**
 - **use**
 - **release**





Deadlock with Mutex Locks / Semaphores

- ❑ The locking tools (e.g., mutex locks, semaphores) are designed to avoid race conditions. However, inappropriate usage of them can lead to deadlocks.

Example?

- ❑ **Deadlock** – A set of processes is deadlocked when **every** process in the set is waiting for the resource that is currently allocated to another process in the set
- ❑ Let S and Q be two semaphores initialized to 1

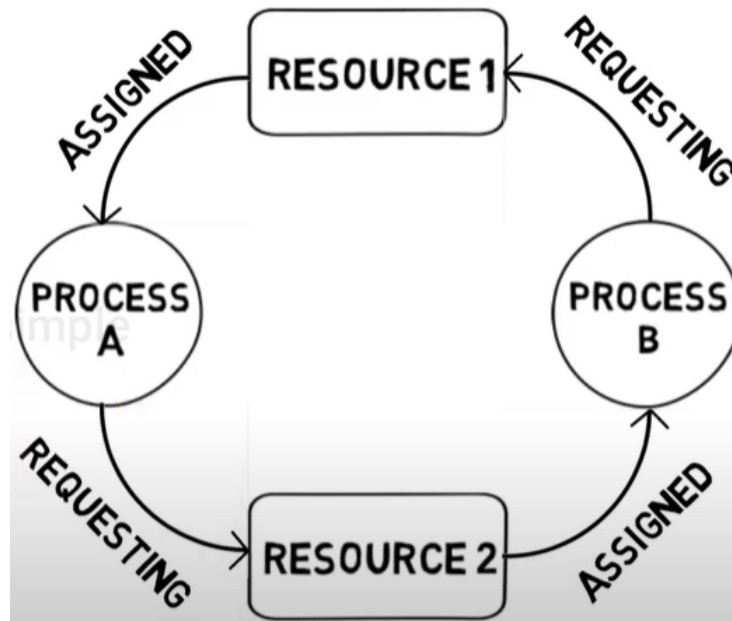
P_0
`wait(S) ;`
`wait(Q) ;`
`...`
`signal(S) ;`
`signal(Q) ;`

P_1
`wait(Q) ;`
`wait(S) ;`
`...`
`signal(Q) ;`
`signal(S) ;`





Deadlock Example





Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- ❑ **Mutual exclusion:** **at least one** resource must be held in a non-sharable mode; that is, only one process at a time can use a resource
- ❑ **Hold and wait:** a process must be holding **at least one** resource and waiting to acquire additional resources held by other processes
- ❑ **No preemption:** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task
- ❑ **Circular wait:** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

The circular-wait condition implies the hold-and-wait condition





Resource-Allocation Graph

Deadlocks can be described more precisely in terms of a **directed graph** called a system **resource-allocation graph**

A set of vertices V and a set of edges E .

□ V is partitioned into two types:

➤ $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system

➤ $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system

□ **request edge** – directed edge $P_i \rightarrow R_j$

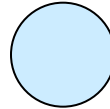
□ **assignment edge** – directed edge $R_j \rightarrow P_i$





Resource-Allocation Graph (Cont.)

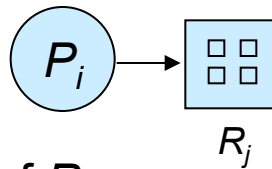
- Process



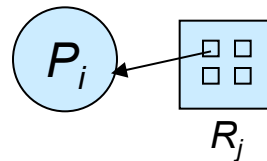
- Resource Type with 4 instances

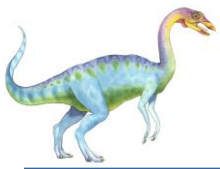


- P_i requests instance of R_j

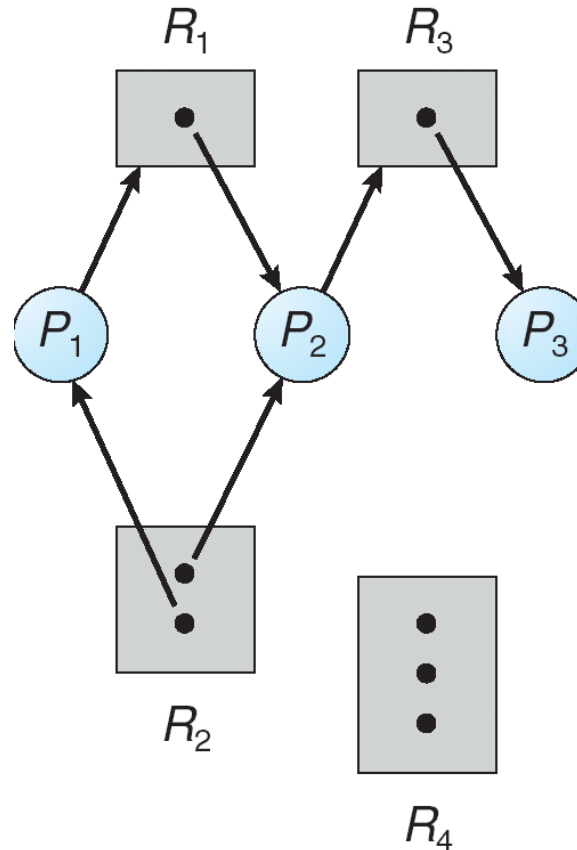


- P_i is holding an instance of R_j





Example of a Resource Allocation Graph

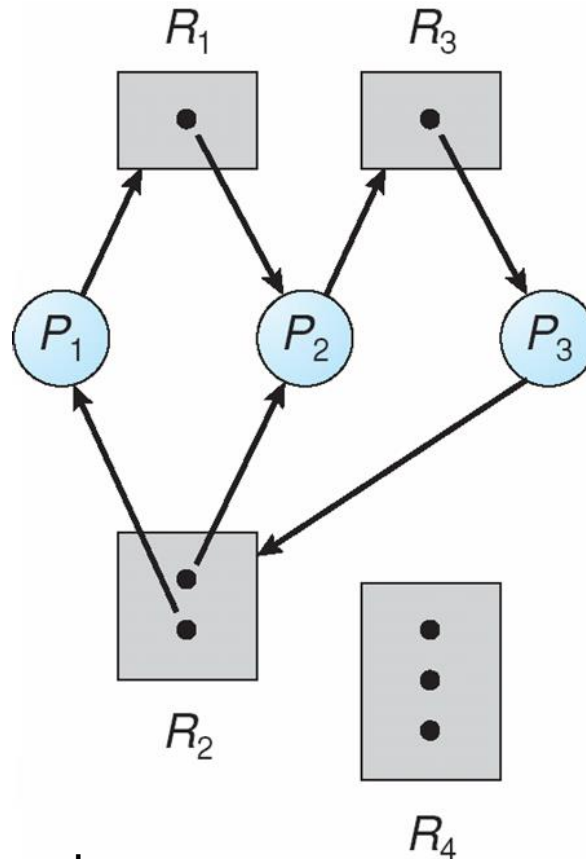


- ❑ Do we have a deadlock here?
- ❑ It can be shown that if the graph contains no cycles, then no process in the system is deadlocked





Resource Allocation Graph With A Deadlock



Processes P_1 , P_2 , and P_3 are deadlocked. Process P_2 is waiting for the resource R_3 , which is held by process P_3 . Process P_3 is waiting for either process P_1 or process P_2 to release resource R_2 . In addition, process P_1 is waiting for process P_2 to release resource R_1 .

Two minimal cycles:

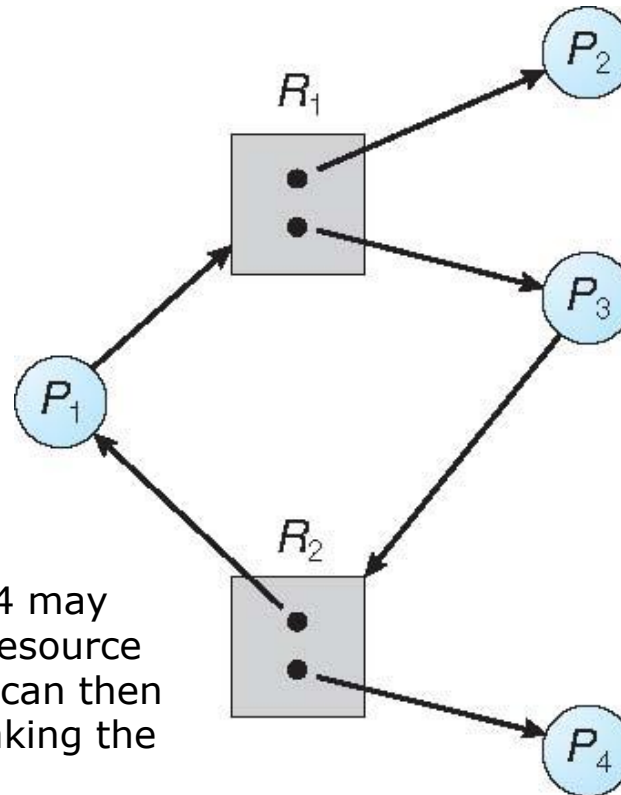
$$\begin{aligned} P_1 &\rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1 \\ P_2 &\rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2 \end{aligned}$$

Do we have a deadlock here?





Graph With A Cycle But No Deadlock



Observe that process P4 may release its instance of resource type R2. That resource can then be allocated to P3, breaking the cycle.

If the graph does contain a cycle, then a deadlock may exist

$$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$

Do we have a deadlock here?





Basic Facts

- ❑ If graph contains no cycles $\Rightarrow$ no deadlock
- ❑ If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock
 - if several instances per resource type, possibility of deadlock





Quiz

- ☐ Assume **3** processes share **4** identical resources, and
- ☐ each allocation or request can obtain only **1** unit at a time.
- ☐ Given that every process requires **at most 2** units of the resource,
- ☐ will a deadlock occur in the system? Explain why.

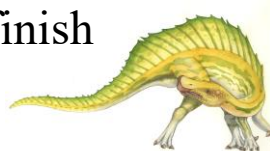
A deadlock will not occur.

Because there are 4 units of the resource for 3 processes, the **pigeonhole principle** guarantees that at least one process must receive two units.

Since a process needs no more than two units in total, that process can finish its execution and then release its resources.

Once those units are freed, the remaining processes can acquire the resources they need and complete their execution.

Therefore, every process will eventually obtain the required resources and finish within a finite time, preventing any deadlock in the system.





Quiz

A computer system has 8 printers shared by K processes. Each process may request up to 3 printers. What is the smallest value of K for which a deadlock could occur in the system?

A well-known result for reusable, non-pre-emptible resources says:

*If each of K processes may request at most M units of a resource and the total number of units is R , then deadlock **cannot** occur if*

$$R \geq K(M - 1) + 1.$$

*Conversely, deadlock **may** occur when $R \leq K(M - 1)$.*

Here $M = 3$ and $R = 8$.

Find the minimum K such that $8 \leq K(3 - 1) = 2K$:

$$2K \geq 8 \implies K \geq 4.$$

With $K = 4$ processes, a deadlock state is possible: each process could hold 2 printers (its "last but one" unit), so all $4 \times 2 = 8$ printers are allocated and every process still needs one more. No printer is free, no process can finish, and deadlock ensues.

With $K = 3$ processes, the maximum simultaneous allocation of 2 printers each uses only 6 printers, leaving 2 free. At least one process can obtain the extra printer it needs, finish, and release its printers, so a deadlock cannot arise.





Quiz

A system contains n identical devices that can be used only one at a time (mutual exclusion).

Three concurrent processes may need up to 3, 4, and 5 devices, respectively.

What is the smallest number n that guarantees the system can never enter a deadlock?

1. In a potential deadlock each process could hold **all but one** of the devices it may ever request.

- Process 1 could hold $3 - 1 = 2$ devices.
- Process 2 could hold $4 - 1 = 3$ devices.
- Process 3 could hold $5 - 1 = 4$ devices.

2. Altogether they could occupy

$$2 + 3 + 4 = 9 \text{ devices.}$$

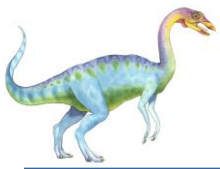
3. If the system had only 9 (or fewer) devices, none would be free, each process would still need one more, and all would wait forever—a deadlock.

4. Therefore the system must have **at least one more device than that worst-case total**, i.e.

$$n \geq 9 + 1 = 10.$$

With 10 devices, at least one device is always free, so some process can obtain its final unit, finish, and release its devices; the others can then proceed, ensuring that deadlock can never occur.





Methods for Handling Deadlocks

➤ Deadlock prevention

- Ensure that **at least one** of the necessary conditions cannot hold

➤ Deadlock avoidance

- Requires that the OS be given **additional information** in advance concerning **which resources a process will request and use during its lifetime**

➤ Detection & Recovery

- Allow the system to **enter** a deadlock state, **detect** it, and then **recover**



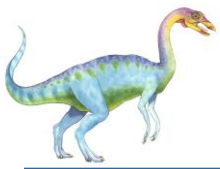


Deadlock Prevention

❑ **Mutual Exclusion** – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources

➤ We cannot prevent deadlocks by denying the mutual-exclusion condition





Deadlock Prevention

- ❑ **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
 - Method 1: Require process to request and be allocated all its resources before it begins execution, **disadvantage?**
 - Method 2: Allow process to request resources only when the process has none allocated to it.
 - Before request, first release all
 - Consider a process that copies data from a DVD driver to a file on disk, sorts the file, and then prints the results to a printer
 - Method 1: request all three resources in the beginning
 - Method 2: request DVD and disk file first; copy from DVD to disk file; release both DVD and file; and then request disk file and printer
 - Low resource utilization; starvation possible, why?





Deadlock Prevention

- Consider a process that copies data from a DVD driver to a file on disk, sorts the file, and then prints the results to a printer
 - Method 1: request all three resources in the beginning
 - Method 2: request DVD and disk file first; copy from DVD to disk file; release both DVD and file; and then request disk file and printer

- Low resource utilization; starvation possible, why?
 - Resource utilization may be low, since resources may be allocated but unused for a long period.
 - A process that needs several popular resources may have to wait indefinitely, because at least one of the resources that it needs is always allocated to some other process.



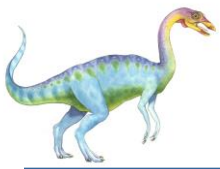


Deadlock Prevention (Cont.)

❑ No Preemption –

- If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are **preempted** (released)
- Preempted resources are added to the list of resources for which the process is waiting
- Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting





Deadlock Prevention (Cont.)

- ❑ **Circular Wait** – impose a total **ordering** of all **resource types**, and require that each process requests resources **in an increasing order** of enumeration
 - $F(\text{tape drive}) = 1; F(\text{disk drive}) = 5; F(\text{printer}) = 12$
 - A process can initially request any number of instances of a resource type---say, R_i . After that, the process can request instances of resource type R_j if and only if $F(R_j) > F(R_i)$
 - Alternatively, we can require that a process requesting an instance of resource type R_j must have released any resources R_i such that $F(R_i) > F(R_j)$

If these two protocols are used, then the circular-wait condition cannot hold





Deadlock Example

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);

    pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
    pthread_mutex_lock(&second_mutex);
    pthread_mutex_lock(&first_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);

    pthread_exit(0);
}
```

If the lock ordering was:

$F(\text{first_mutex}) = 1$

$F(\text{second_mutex}) = 5$

The thread_two could not
request the locks out of order





Deadlock Avoidance

Requires that the system has some **additional information** available, about **how resources are to be request**

- ❑ Simplest and most useful model requires that each process declare the **maximum number** of resources of each type that **it may need**
- ❑ The deadlock-avoidance algorithm **dynamically examines the resource-allocation state** to ensure that there can **never be a circular-wait condition**
- ❑ Resource-allocation *state* is defined by the number of **available** and **allocated** resources, and the **maximum demands** of the processes

Need priori information





Deadlock Avoidance



Process P will request first the tape drive and then the printer before releasing both resources?

Whereas process Q will request first the printer and then the tape drive?

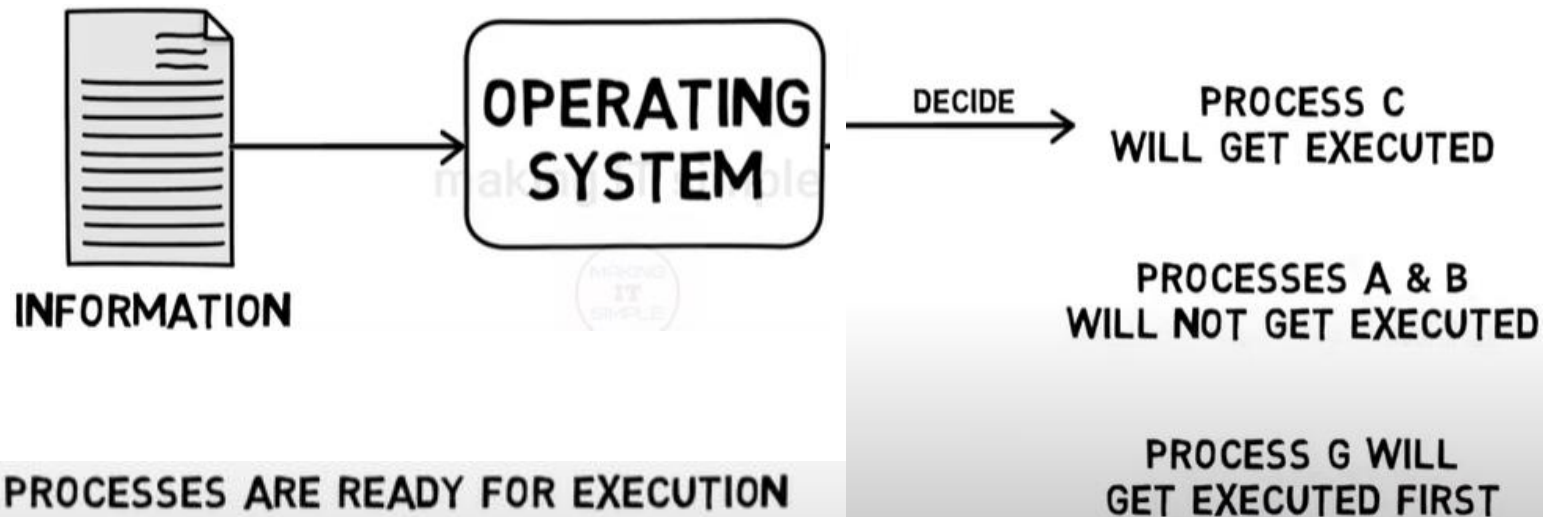
With this knowledge of the **complete sequence of requests and releases for each process**, the system can decide for each request **whether or not the process should wait** in order to avoid a possible future deadlock.





Deadlock Avoidance

Just avoiding problem when it is about to happens



1) WHICH PROCESSES ARE READY FOR EXECUTION

2) WHAT RESOURCES ARE REQUIRED FOR PROCESSES

3) HOW MUCH OF TIME WILL THE PROCESS HOLD RESOURCES





Safe State

- When a thread requests an available resource, system must decide if immediate allocation leaves the system in a safe state
- System is in **safe state** if there exists a sequence $\langle T_1, T_2, \dots, T_n \rangle$ of ALL the threads in the systems such that for each T_i , the resources that T_i can still request can be satisfied by currently available resources + resources held by all the T_j , with $j < i$
- That is:
 - If T_i resource needs are not immediately available, then T_i can wait until all T_j have finished
 - When T_j is finished, T_i can obtain needed resources, execute, return allocated resources, and terminate
 - When T_i terminates, T_{i+1} can obtain its needed resources, and so on

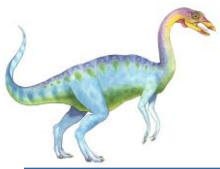




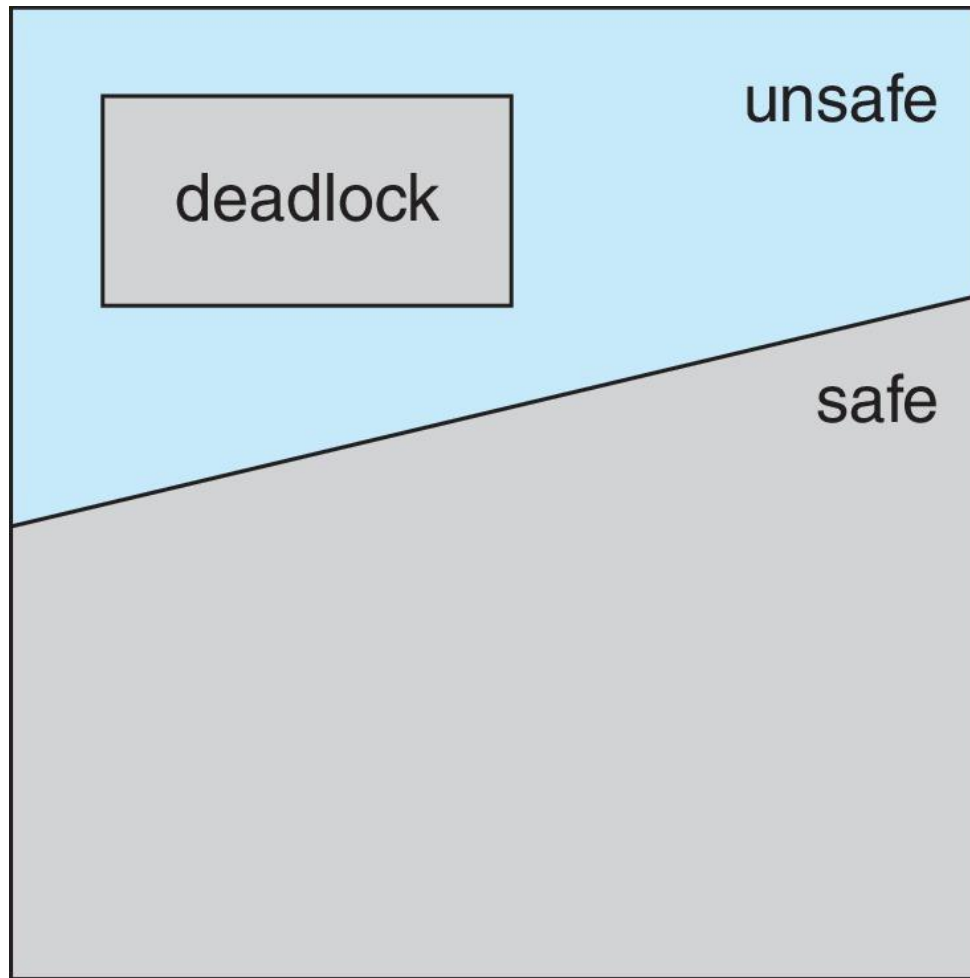
Basic Facts

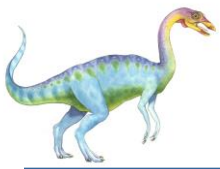
- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.





Safe, Unsafe, Deadlock State



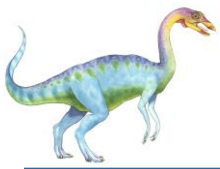


Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph

- Multiple instances of a resource type
 - Use the Banker's Algorithm

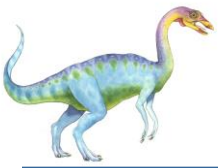




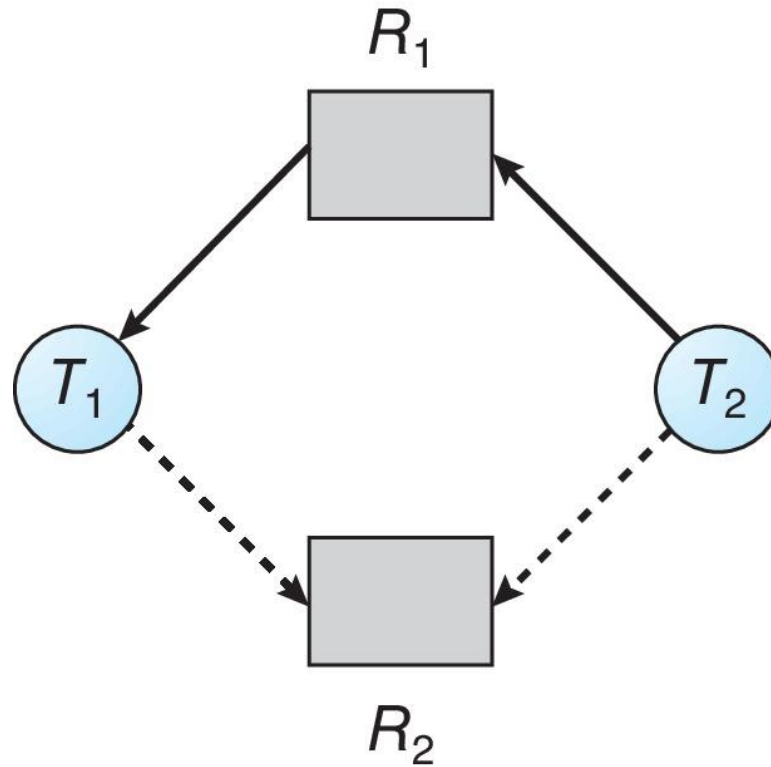
Resource-Allocation Graph Scheme

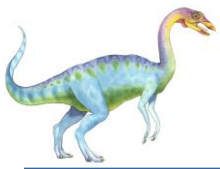
- **Claim edge** $T_i \rightarrow R_j$ indicated that process T_j may request resource R_j ; represented by a dashed line
- Claim edge converts to request edge when a thread requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the thread
- When a resource is released by a thread, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system



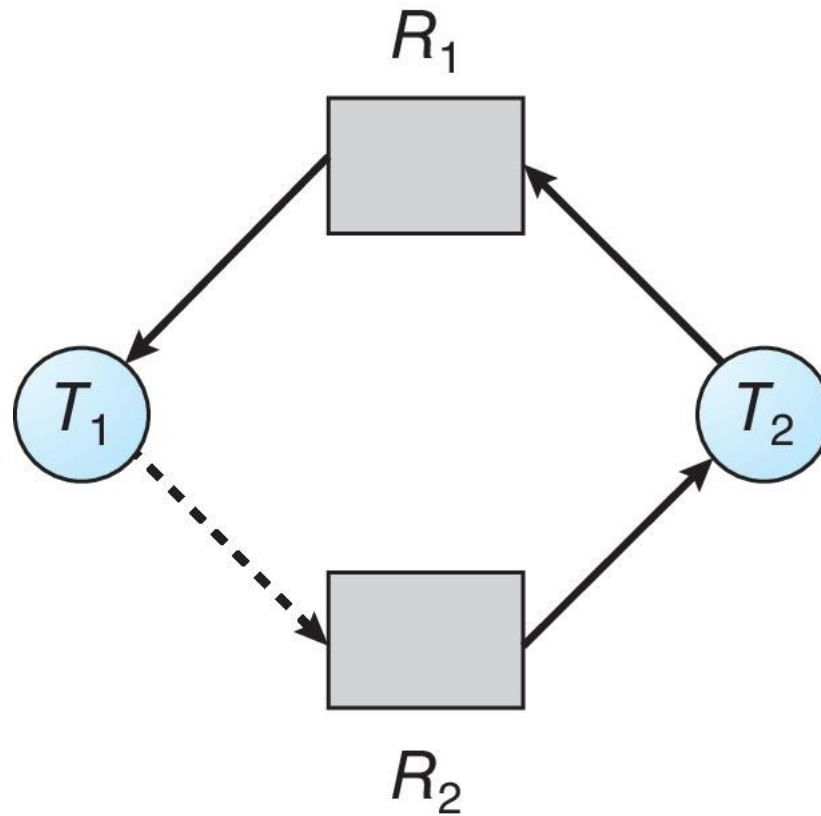


Resource-Allocation Graph





Unsafe State In Resource-Allocation Graph





Resource-Allocation Graph Algorithm

- Suppose that thread T_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph





Banker's Algorithm

- ❑ A resource allocation and deadlock avoidance algorithm
- ❑ Tests for safety by **simulating the allocation** for predetermined **maximum possible** amounts of all resources
- ❑ Then **makes a check** to test for **possible activities**, before deciding whether allocation should be allowed to continue.





Banker's Algorithm

- Multiple instances of resources
- Each thread must a priori claim maximum use
- When a thread requests a resource, it may have to wait
- When a thread gets all its resources it must return them in a finite amount of time





Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i, j] = k$, then process T_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i, j] = k$ then T_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i, j] = k$, then T_i may need k more instances of R_j to complete its task

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$





Safety Algorithm

1. Let **Work** and **Finish** be vectors of length m and n , respectively.
Initialize:

Work = Available

Finish [i] = false for $i = 0, 1, \dots, n-1$

2. Find an i such that both:

(a) **Finish [i] = false**

(b) **Need_i ≤ Work**

If no such i exists, go to step 4

3. **Work = Work + Allocation_i**
Finish[i] = true
go to step 2

4. If **Finish [i] == true** for all i , then the system is in a safe state





Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process T_i . If **$Request_i[j] = k$** then process T_i wants k instances of resource type R_j

1. If **$Request_i \leq Need_i$** , go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If **$Request_i \leq Available$** , go to step 3. Otherwise T_i must wait, since resources are not available
3. Pretend to allocate requested resources to T_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- If safe $\Rightarrow$ the resources are allocated to T_i
- If unsafe $\Rightarrow T_i$ must wait, and the old resource-allocation state is restored





Banker's Algorithm

- ❑ Considering a system with 5 processes, and 3 resources of type A, B, C
 - ❑ A: 10 instances
 - ❑ B: 5 instances
 - ❑ C: 7 instances
- ❑ Suppose at time t_0 following snapshot of the system has been taken:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2
P ₁	2 0 0	3 2 2	
P ₂	3 0 2	9 0 2	
P ₃	2 1 1	2 2 2	
P ₄	0 0 2	4 3 3	





Banker's Algorithm

Process	Allocation	Max	Available
	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2
P ₁	2 0 0	3 2 2	
P ₂	3 0 2	9 0 2	
P ₃	2 1 1	2 2 2	
P ₄	0 0 2	4 3 3	

Question1. What will be the content of the **Need matrix**?

$$\text{Need} = \text{Max} - \text{Allocation}$$

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1





Banker's Algorithm

Question2. Is the system in a safe state? If Yes, then what is the safe sequence?

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	3	3	2
P ₁	2	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

For P₀: need = 7 4 3
 available = 3 3 2

 need > available

P₀ must wait





Banker's Algorithm

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	3	3	2
P ₁	2	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

For P₁: need = 1 2 2
 available = 3 3 2

 need < available

P₁ can be allocated to
resources

After P₁ is finished, all allocated
resources must be released

$$\text{available} = 3 \ 3 \ 2 + 2 \ 0 \ 0 = 5 \ 3 \ 2$$





Banker's Algorithm

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	5	3	2
P₁	2	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

For P₂: need = 6 0 0
 available = 5 3 2

 need > available

P₂ must wait





Banker's Algorithm

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	5	3	2
P₁	2	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

For P₃: need = 0 1 1
 available = 5 3 2

 need < available

P₃ can be allocated to
resources

After P₃ is finished, all allocated
resources must be released

available = 5 3 2 + 2 1 1 = 7 4 3





Banker's Algorithm

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	7	4	3
P₁	2	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

For P₄: need = 4 3 1
 available = 7 4 3

 need < available

P₄ can be allocated to
resources

After P₄ is finished, all allocated
resources must be released

available = 7 4 3 + 0 0 2 = 7 4 5





Banker's Algorithm

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	7	4	5
P₁	2	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P₃	2	1	1	2	2	2			
P₄	0	0	2	4	3	3			

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

For P₀: need = 7 4 3
 available = 7 4 5

 need < available

P₀ can be allocated to
resources

After P₀ is finished, all allocated
resources must be released

available = 7 4 5 + 0 1 0 = 7 5 5





Banker's Algorithm

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P₀	0	1	0	7	5	3	7	5	5
P₁	2	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P₃	2	1	1	2	2	2			
P₄	0	0	2	4	3	3			

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

For P₂: need = 6 0 0
 available = 7 5 5

 need < available

P₂ can be allocated to
resources

After P₂ is finished, all allocated
resources must be released

$$\text{available} = 7 \ 5 \ 5 + 3 \ 0 \ 2 = 10 \ 5 \ 7$$

Hence, the system is in safe state!
The safe sequence is P₁, P₃, P₄, P₀, P₂





Quiz

Answer: First, we calculate the *Need* array:

	<i>Allocation</i>	<i>Max</i>	Need	Available
P_0	5	10	5	12
P_1	2	4	2	
P_2	3	9	6	

Keep in mind that we always use a top-down scan. Because P_0 's *Need* = 5 $\leq$ *Available* = 12, P_0 can run and returns its allocated 5, making the new *Available* = 12 + 5 = 17.

	<i>Allocation</i>	<i>Max</i>	Need	Available
P_0		10		17
P_1	2	4	2	
P_2	3	9	6	

Then, P_1 can run because its *Need* = 2 $\leq$ *Available* = 17. After P_1 finishes its work, its *Allocation* = 2 is returned to *Available*, and the new *Available* = 17 + 2 = 19.

	<i>Allocation</i>	<i>Max</i>	Need	Available
P_0		10		19
P_1		4		
P_2	3	9	6	

Finally, because P_2 's *Need* = 3 $\leq$ *Available* = 17, P_2 can run. As a result, this system is in a safe state and $\langle P_0, P_1, P_2 \rangle$ is a safe sequence. ■

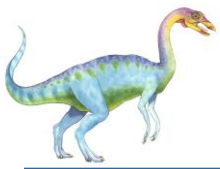




Deadlock Detection

- ❑ Allow system to enter deadlock state
- ❑ Detection algorithm
 - An algorithm that examines the state of the system to determine whether a deadlock has occurred
- ❑ Recovery scheme
 - Option 1: Process Termination-----abort one or more processes to break the circular wait
 - Option 2: Process Preemption-----preempt some resources from one or more of the deadlocked processes





Recovery from Deadlock: Process Termination

- ❑ Abort all deadlocked processes
 - What's drawback?
 - Break, but at great expense
- ❑ Abort one process at a time until the deadlock cycle is eliminated
 - What's drawback?
 - Incurs considerable overhead, deadlock-detection algorithm invoked frequently
- ❑ In which order should we choose to abort?
 1. Priority of the process
 2. How long process has computed, and how much longer to completion
 3. Resources the process has used
 4. Resources process needs to complete
 5. How many processes will need to be terminated
 6. Is process interactive or batch?

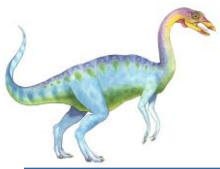




Recovery from Deadlock: Resource Preemption

- ❑ Successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken
- ❑ **Selecting a victim** – minimize cost
 - Number of resources is holding; amount of time has thus far consumed
- ❑ **Rollback** – return to some safe state, restart process from that state
 - Hard to determine, so total rollback
- ❑ **Starvation** – same process may always be picked as victim, include number of rollback in cost factor





Deadlock prevention

Deadlock prevention by **ordering the resource requesting**





Deadlock prevention

Topic: Deadlock prevention by **ordering the resource requesting**

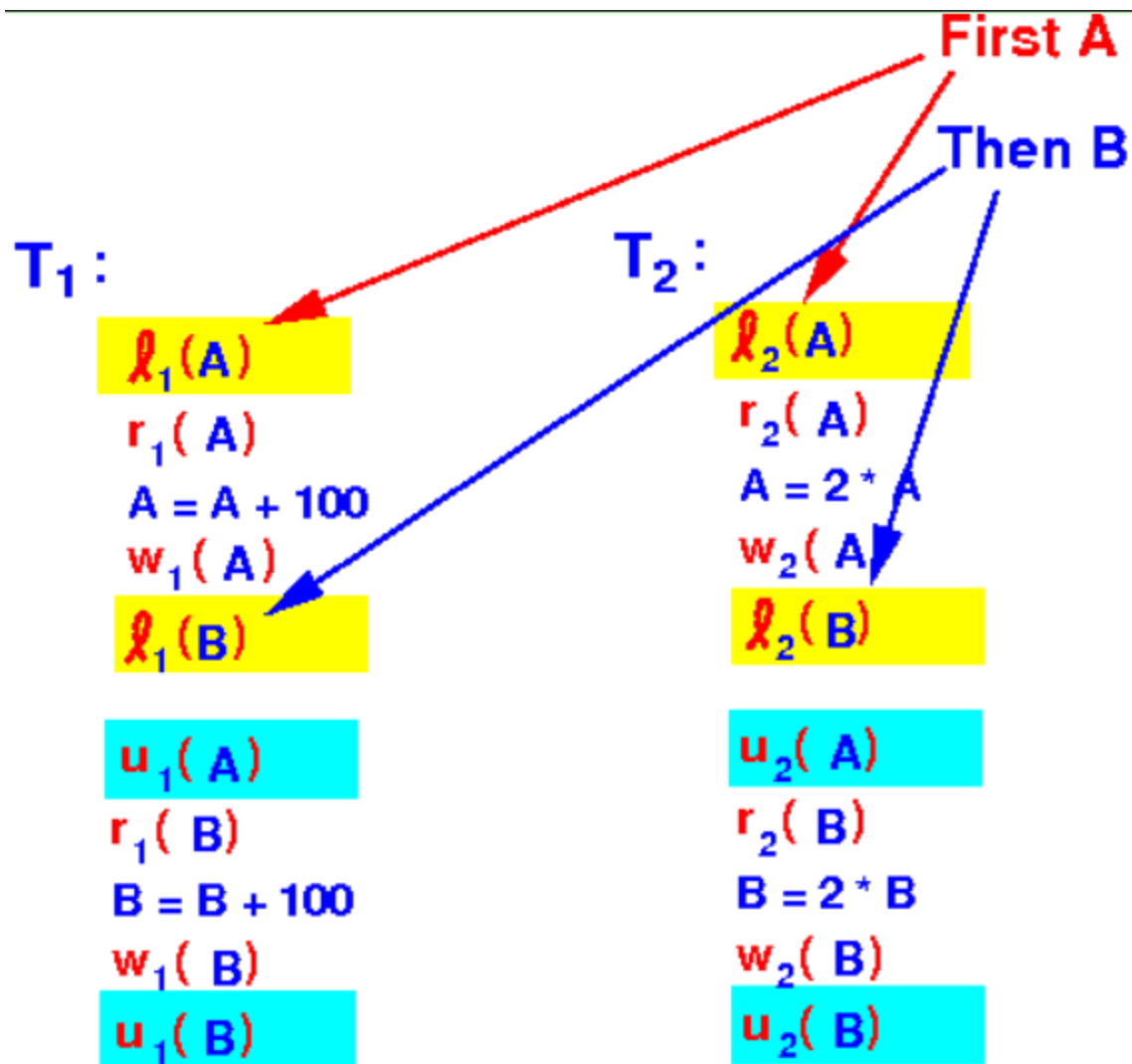
- ❑ Deadlock develops through "**bad**" **ordering** of locking operations
- ❑ "**Good**" **ordering** of **locking operation**:

Processes T1 and T2 lock shared variables A and B in the **same order**:





Deadlock prevention





Deadlock prevention

Topic: Deadlock prevention by **ordering the resource requesting**

- ❑ Deadlock develops through "**bad**" **ordering** of locking operations
- ❑ "**Good**" **ordering** of **locking operation**:

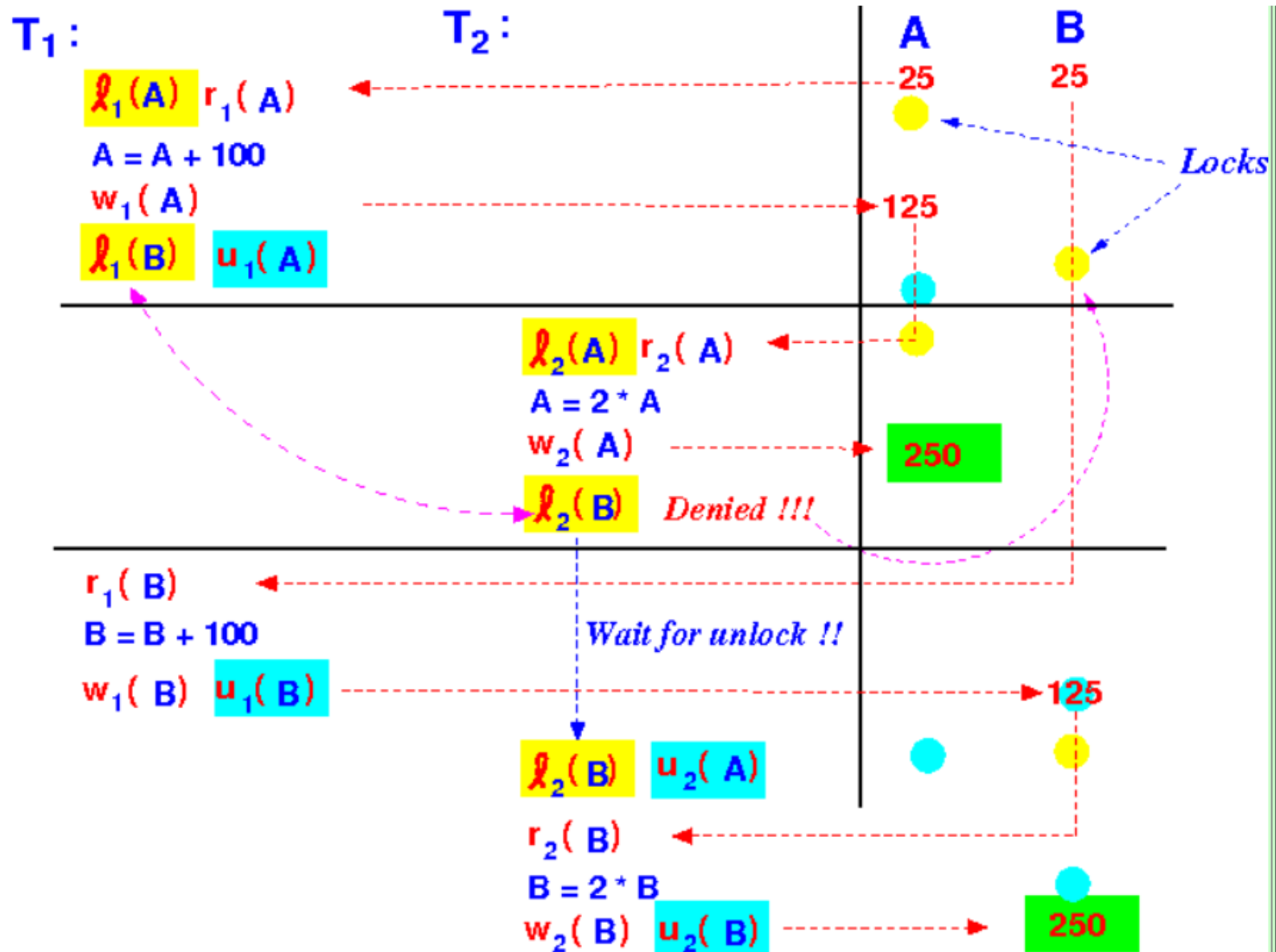
Processes T1 and T2 lock shared variables A and B in the **same order**:

Interleaved execution of processes T1 and T2 does **not** result in deadlock:





Deadlock prevention





Deadlock prevention

Topic: Deadlock prevention by **ordering the resource requesting**

- ❑ Deadlock develops through "**bad**" **ordering** of locking operations
- ❑ "**Good**" **ordering** of **locking operation**:

Processes T1 and T2 lock shared variables A and B in the **same order**:

Interleaved execution of processes T1 and T2 does **not** result in deadlock:

- ❑ "**Bad**" **ordering** of locking operation:

Processes T1 and T2 locks shared variables A and B in the **opposite order**:





Deadlock prevention

T_1 :

$\ell_1(A)$

$r_1(A)$

$A = A + 100$

$w_1(A)$

$\ell_1(B)$

$u_1(A)$

$r_1(B)$

$B = B + 100$

$w_1(B)$

$u_1(B)$

T_2 :

$\ell_2(B)$

$r_2(B)$

$B = 2 * B$

$w_2(B)$

$\ell_2(A)$

$u_2(B)$

$r_2(A)$

$A = 2 * A$

$w_2(A)$

$u_2(A)$

*Lock order
on A and B
is reversed !*





Deadlock prevention

Topic: Deadlock prevention by **ordering the resource requesting**

- ❑ Deadlock develops through "**bad**" **ordering** of locking operations
- ❑ "**Good**" **ordering** of **locking operation**:

Processes T1 and T2 lock shared variables A and B in the **same order**:

Interleaved execution of processes T1 and T2 does **not** result in deadlock:

- ❑ "**Bad**" **ordering** of locking operation:

Processes T1 and T2 locks shared variables A and B in the **opposite order**:

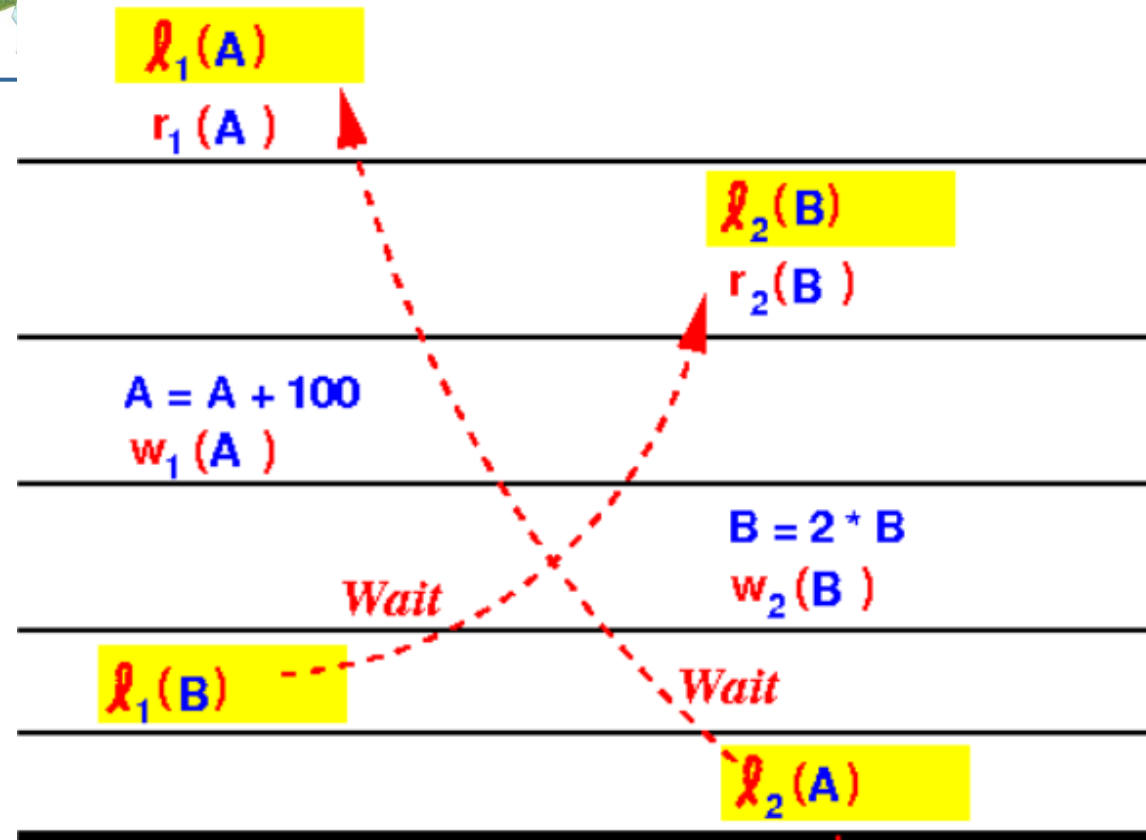
- ❑ **An execution that leads to deadlock:**





T_1 :

T_2 :





Deadlock prevention

Topic: Deadlock prevention by **ordering the resource requesting**

❑ **Preventing deadlock** by imposing an **ordering** on the shared variables

Example ordering:

- Consider shared variables (resources) as a binary number

Example ordering:

- Processes must request locks on shared variables in the **ordering** of variables





Deadlock prevention

A process T wants to lock the shared resources:

Printer 567
Disk 789
File 123

Lock request **order** made by transaction T:

File 123
Printer 567
Disk 789





Deadlock prevention

Topic: Deadlock prevention by **ordering the resource requesting**

❑ The **cause** of the deadlock:

The transactions T1 and T2 have each **locked** the shared variables in **reverse order**:

- **T₁** locks **A** $\Rightarrow$ **B**
- **T₂** locks **B** $\Rightarrow$ **A**

❑ **Preventing deadlock** by imposing an **ordering** on the shared variables

Example ordering:

- Consider shared variables (resources) as a binary number

Example ordering:

- Processes must request locks on shared variables in the **ordering** of variables





Deadlock prevention

- ❑ Theorem: **Ordered locking** on shared resources will prevent deadlock

Claim:

If all processes request **locks** on shared resources in the **order** of the resources, then **deadlock is not possible**.

Proof: by contradiction:

Suppose that:

- Processes T1, T2, ..., Tn request **locks** on shared resources according to the **ordering** of the database elements and
- Transactions T1, T2, ..., Tn are **deadlocked**

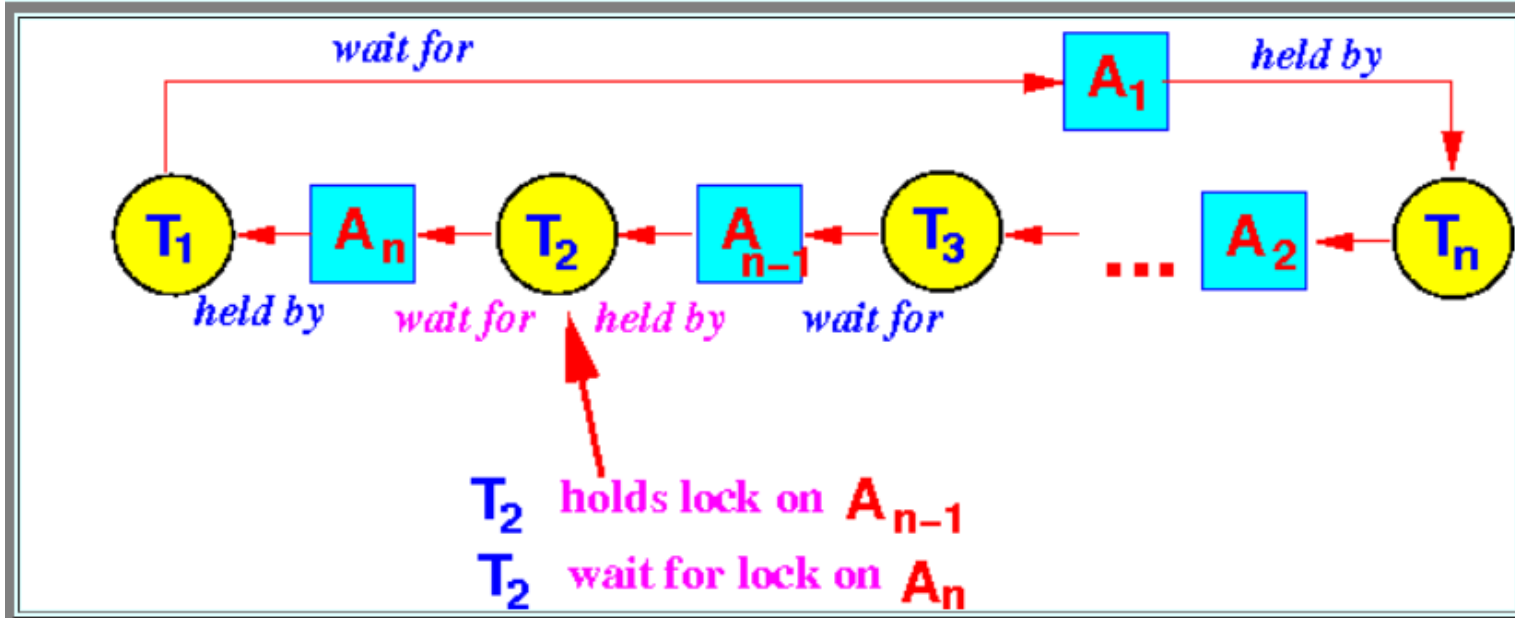




Deadlock prevention

Without loss of generality, let's assume that

The **wait-for graph**



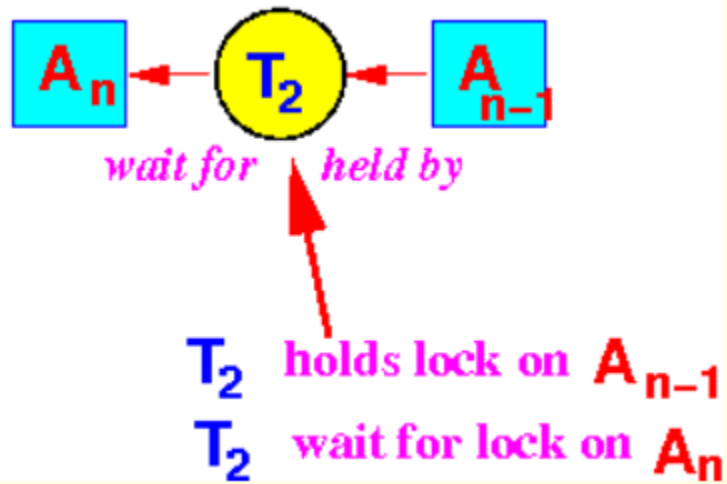
- T1 has locked some variable A_n and T2 is waiting for a lock on A_n
- T2 has locked some variable A_{n-1} and T3 is waiting for a lock on A_{n-1}
- T3 has locked some variable A_{n-2} and T4 is waiting for a lock on A_{n-2}
- ...
- T_{n-1} has locked some variable A_2 and T_n is waiting for a lock on A_2
- T_n has locked some variable A_1 and T1 is waiting for a lock on A_1



Deadlock prevention

Consider the shared variables that **process T2 locks**:

- T2 has **locked** the variable A_{n-1} (**first**) and
- T2 is **waiting** (trying to lock) for the variable A_n (**next**)



Since process T2 will **lock** variable according to their **order**, we conclude that:

$$\blacksquare A_{n-1} < A_n$$

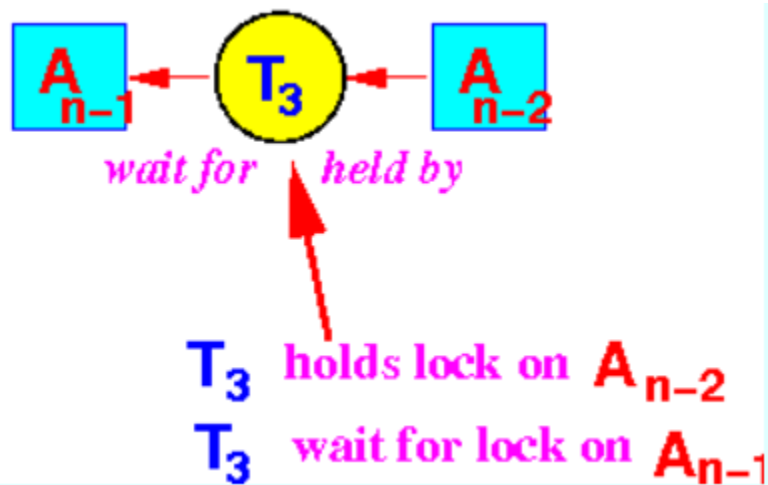




Deadlock prevention

Consider the shared variables that **process T3 locks**:

- T3 has **locked** the variable A_{n-2} (**first**) and
- T3 is **waiting** (trying to lock) for the variable A_{n-1} (**next**)



Since process T3 will **lock** variable according to their **order**, we conclude that:

$$\blacksquare A_{n-2} < A_{n-1}$$

Therefore, we have that:

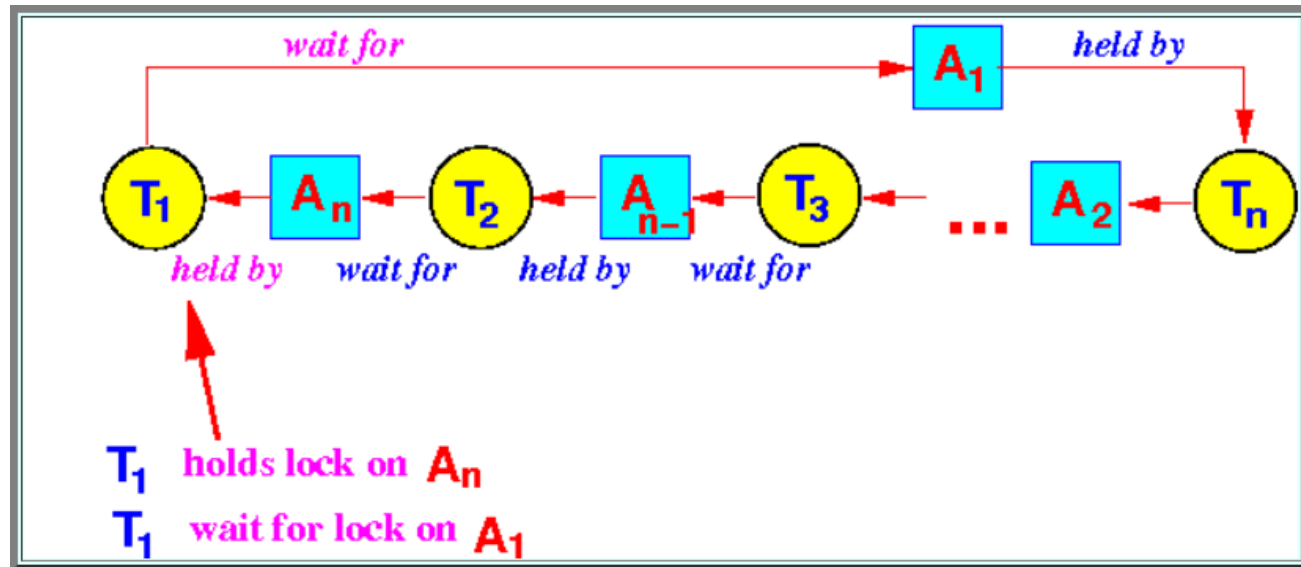
$$\blacksquare A_1 < A_2 < \dots < A_{n-1} < A_n$$





Deadlock prevention

But we also have:

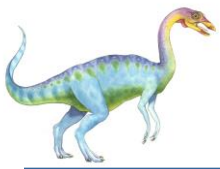


- T_1 has **locked** the A_n and
- T_1 is **waiting** for the A_1

we conclude that:

- $A_n < A_1$





Deadlock prevention

Therefore, we have that:

$$\blacksquare A_1 < A_2 < \dots < A_{n-1} < A_n$$

we conclude that:

$$\blacksquare A_n < A_1$$

$$\blacksquare A_n < A_1 < A_2 < \dots < A_{n-2} < A_{n-1} < A_n$$

And we have a contradiction:

$$\blacksquare A_n < A_n$$

Therefore:

- The assumption that n processes can be involved in a deadlock is wrong
- So deadlock is not possible





Deadlock prevention

Preventing deadlock with **timestamps**: the **wait-die method**





Wait-Die Method

❑ Timestamped processes:

- Each process is assigned a **unique increasing timestamp**

▪ $T_1, T_2, T_3, \dots$

- Important fact:

An **earlier process** receives a **smaller timestamp**

❑ Purpose of the time stamp

To give **older processes** (= processes with **smaller timestamps**) a **higher priority** over "younger" processes





Wait-Die Method

❑ The **Wait-die** locking rule:

- When an **older process** tries to lock a variable that has been locked by a **younger process**

The **older process** will **wait** until the lock is released (by the **younger process**)

- When a **younger process** tries to lock a variable that has been locked by an **older process**

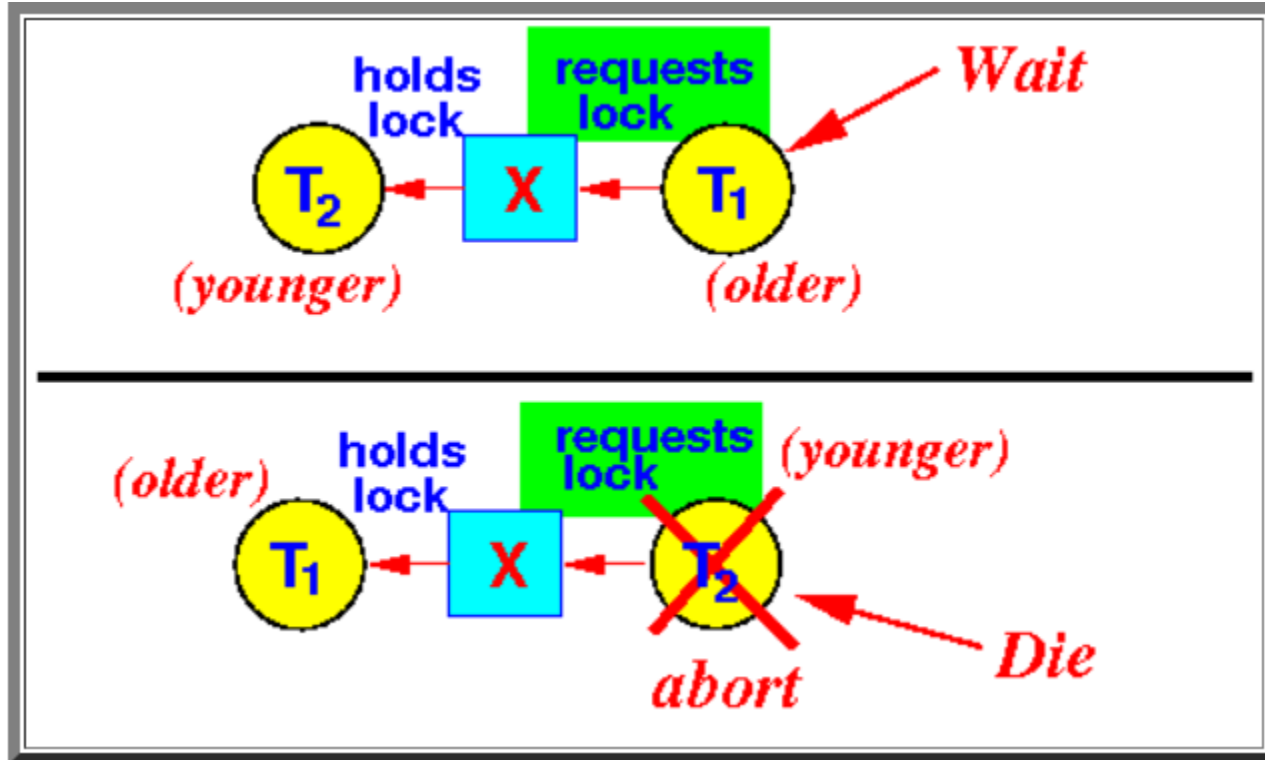
The **younger process** will **die (= abort)**





Wait-Die Method

□ The **Wait-die** locking rule:

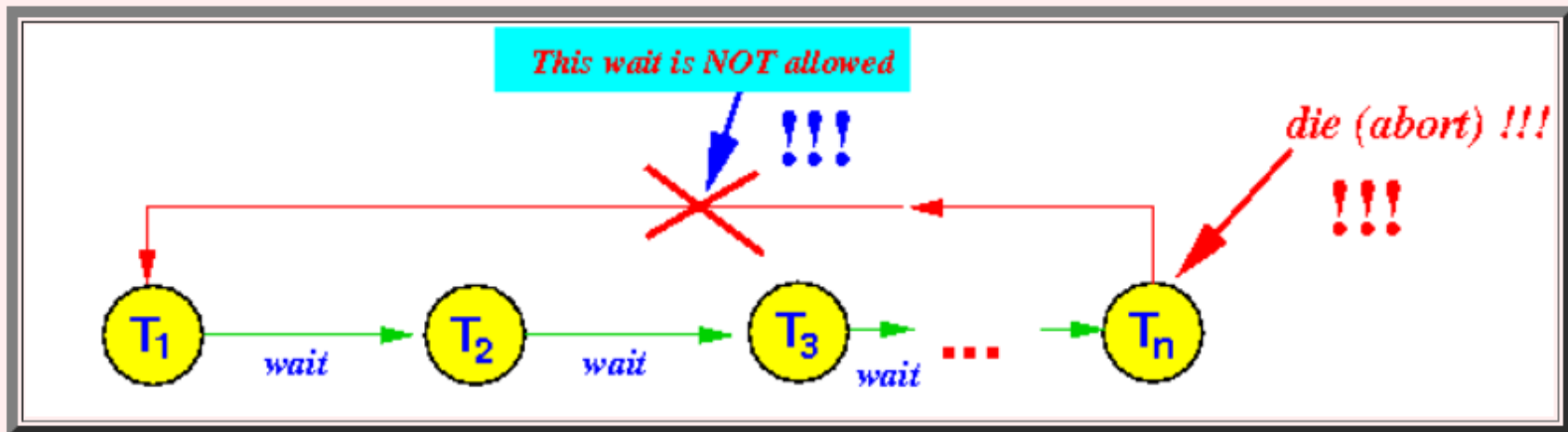




Wait-Die Method

❑ The **Wait-die** locking rule will **prevent** the development of **deadlock** because:

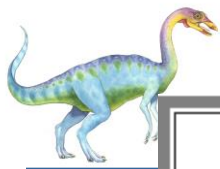
- T_n cannot wait for T_1 :



So there **cannot** be a **cycle** in the **wait-for graph**:

- **No deadlock possible !!!**

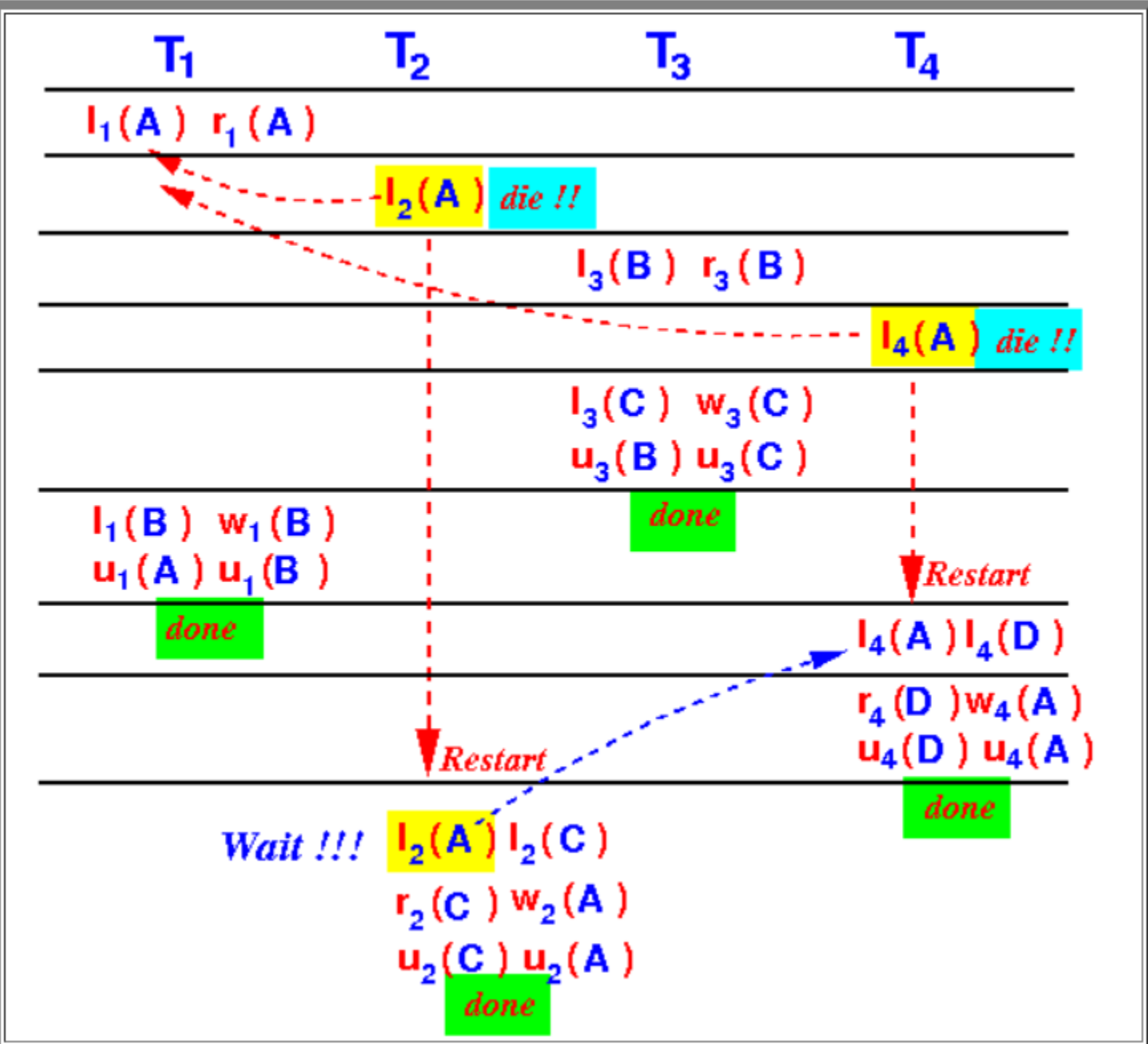




Wait Die Method

Ex

Ex





Wait-Die Method

❑ Important note:

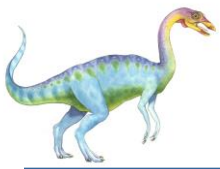
- When a process is **aborted** and **restarts**

The process will **retain** its (old) **timestamp** !!!

- Therefore:

Eventually, a process will become the **"oldest"** process and will **complete execution** !!!





Deadlock prevention

Preventing deadlock with **timestamps**: the **wound-wait method**





Wound-Wait Method

❑ The **Wound-wait** locking rule:

- When an **older process** tries to lock a variable that has been locked by a **younger process**

The **older process** will **wound (= kill / abort)** the **younger process**)

- When a **younger process** tries to lock a variable that has been locked by an **older process**

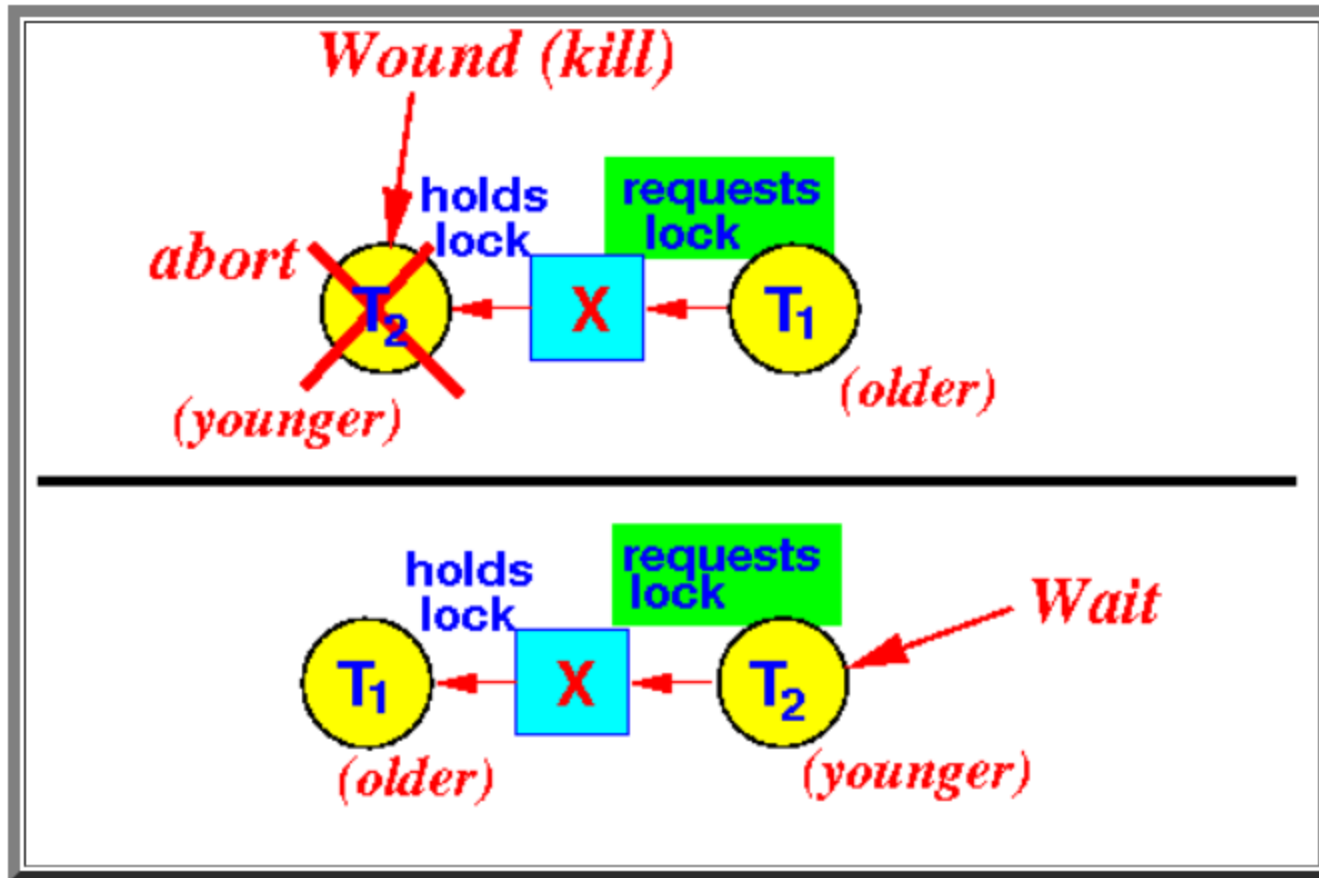
The **younger process** will **wait** until the **lock is released** (by the **older process**)





Wound-Wait Method

□ The **Wound-wait** locking rule:

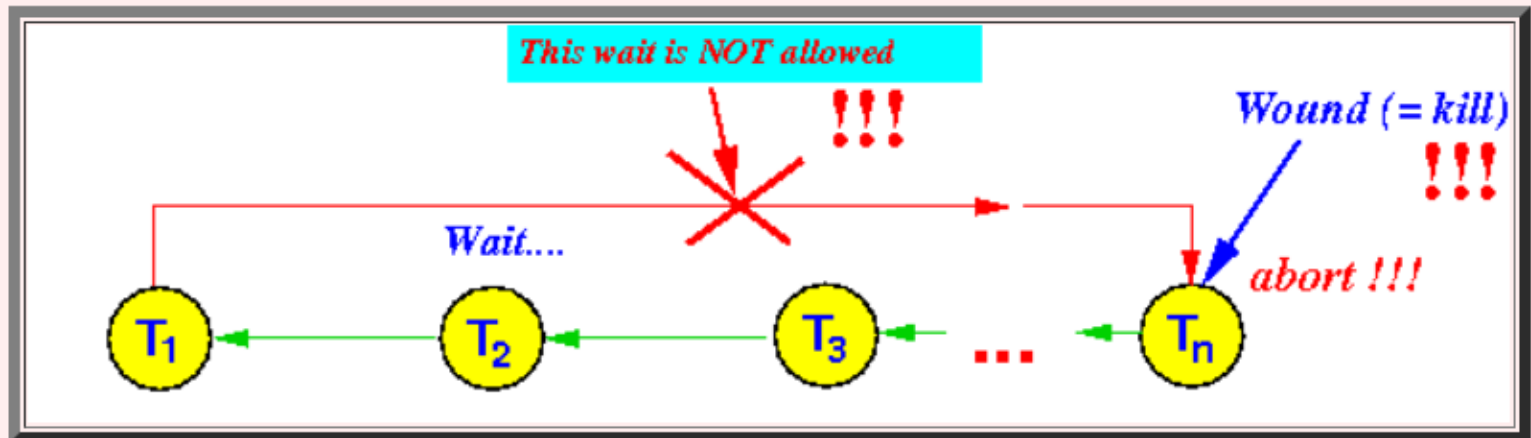




Wound-Wait Method

- ❑ The **Wound-wait** locking rule will **prevent (forbid)** the development of **deadlock** because:

- T_1 will **wound (= kill/abort)** the (younger) T_n :



(The **wait direction** in **wound-wait** is **opposite** from a **wait-die locking rule** !!!)

- In both schemes (**wait-die** and **wound-wait**), the **younger process** will get **aborted**
- The **older transaction** is **not** be **aborted** !!!



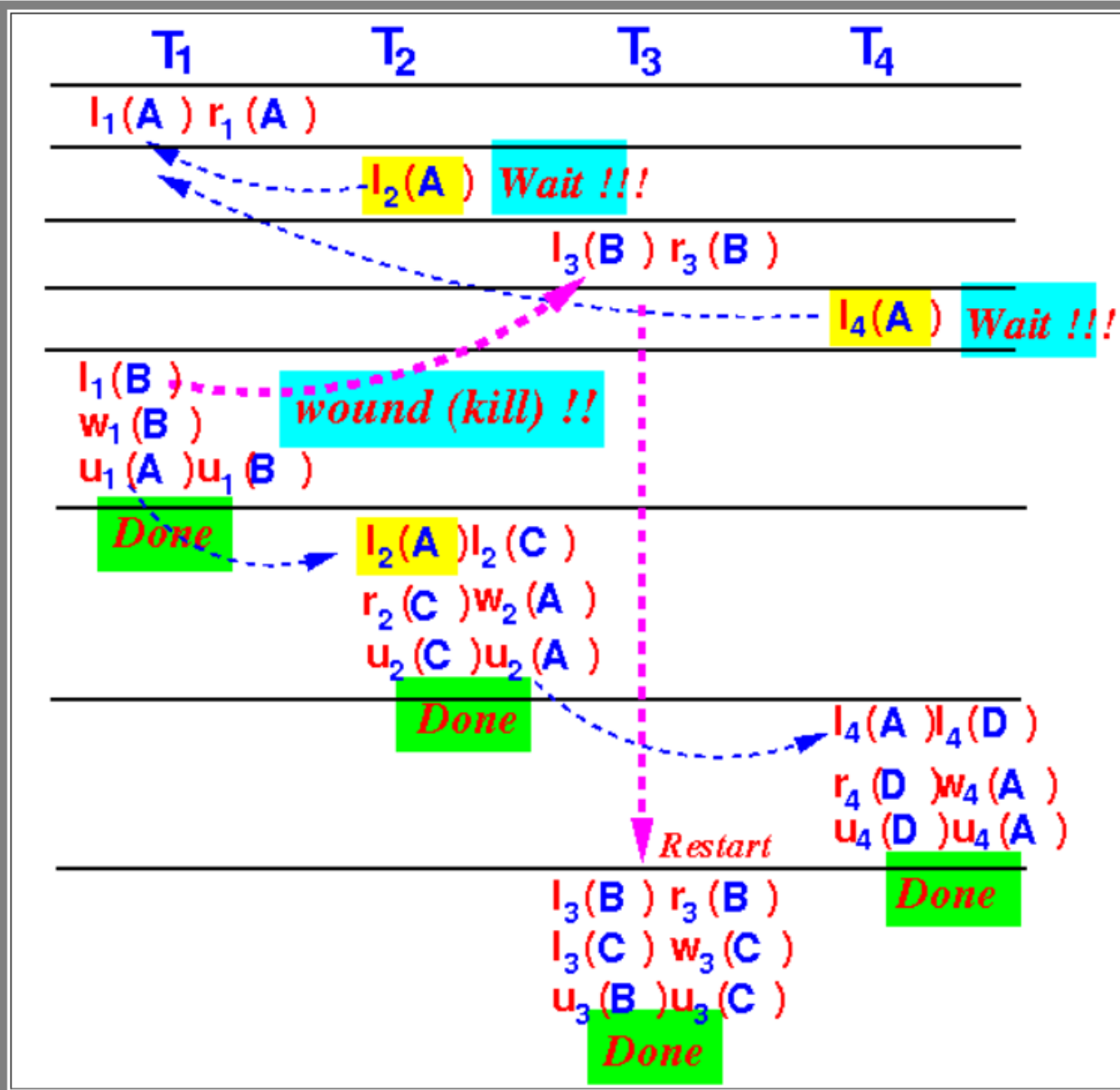


Wound-Wait Method

Example

T₁
T₂
T₃
T₄

Example

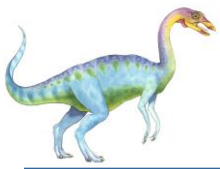




Deadlock prevention

Comparing the wait-die and wound-wait schemes





Comparing

❑ Similarity:

- In both the **wait-die** and the **wound-wait** scheme

The **older process** will **"win"** over the **younger process**

- Both schemes are **fair**

When processes **restart**, they keep their **timestamps**

Eventually, **the aborted (younger) processes** will become the **oldest processes** in the system





Comparing

❑ Difference:

➤ Wait-die:

The younger processes are killed when:

It (= the younger process) makes a request for a lock being held by an older process

➤ Wound-die:

The younger processes are killed when:

An older process makes a request for a lock being held by the younger processes





Comparing

❑ The cost of aborting (younger/older) transactions:

➤ Younger processes (having just started running) will typically:

hold **fewer** number of locks and
has **read/written fewer** number of variables

➤ Older processes (having run longer) will typically

hold **higher** number of locks and
has **read/written higher** number of variables

Older processes (= that started earlier) are more expensive (costly) to be aborted !!!!





Comparing

Which method has higher aborting rate?





Comparing

- ❑ Comparing the **abort rates** of the **wait-die** and **wound-wait** schemes:

- The **number of processes** that will be **aborted** in the **wound-wait** method will be

lower than **number of processes** that will be **aborted** in the **wait-die** method

Reasons:

- ❑ A **younger process** **starts later** than an **older process**:

Wound-wait:

T_1

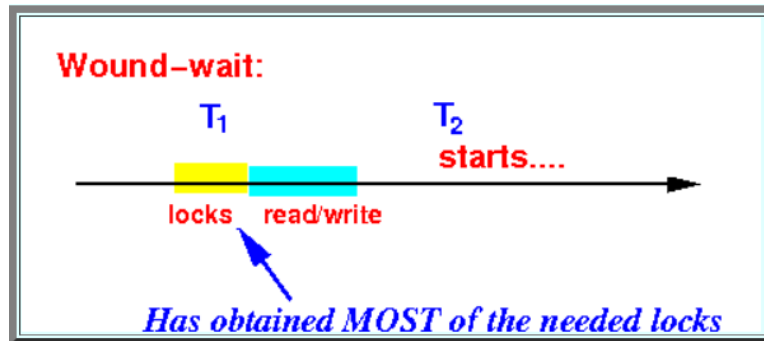
T_2



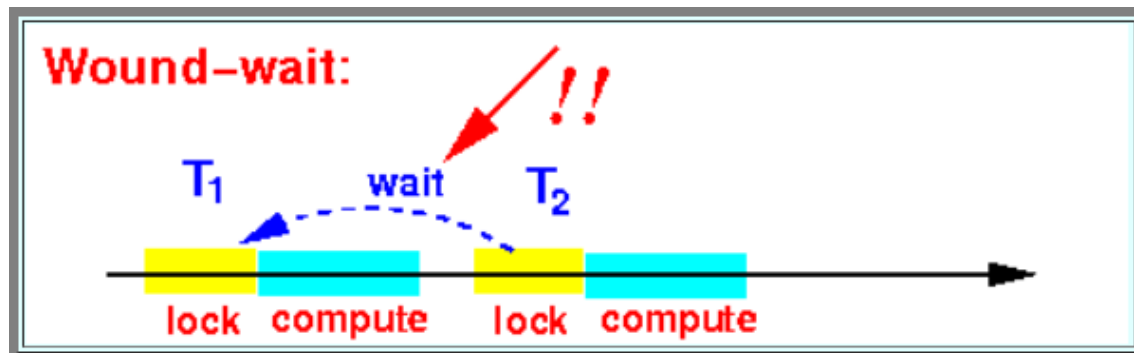


Comparing

- By the time that the younger process T2 starts, the older process T1 will have obtained most of their locks:



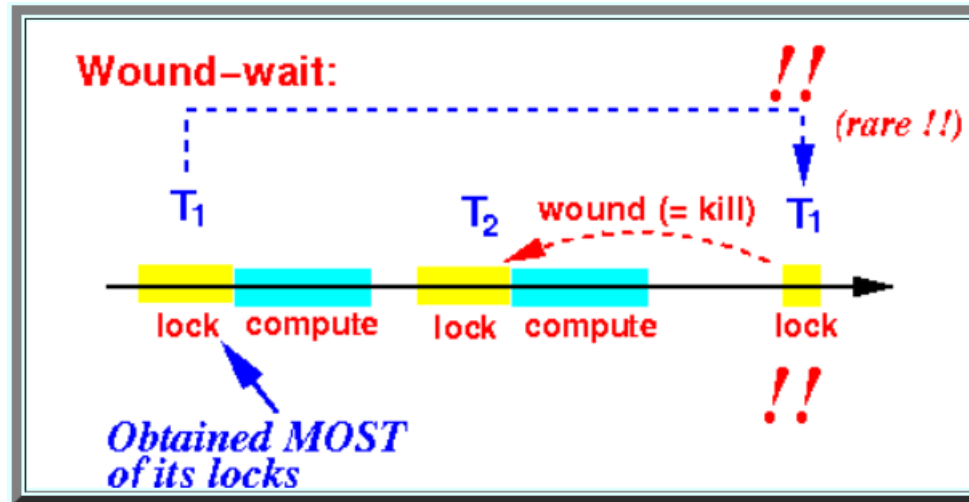
- A younger (just starting) process T2 will make many lock requests !!
- In the wound-wait method, a younger (just starting) process T2 will wait (and not abort) if T2 requests a lock held by the older process T1:





Comparing

- ❑ An older process (T1) that requests a **lock** can **abort** a younger process (T2)



However:

- The event that older processes (T1) will request a **lock** is **rare**
- (because older processes (T1) have obtained **most** of their locks already)

Therefore:

- The **number of aborts** in **wound-wait** is relatively **low**





Comparing

- ❑ Wait-die method will have more **aborts**:
- ❑ A **younger process starts later** than an **older process**:

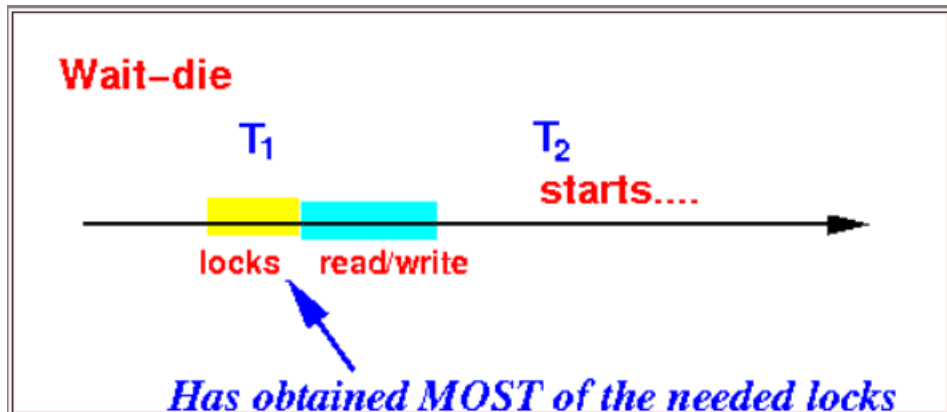
Wait-die

T_1

T_2

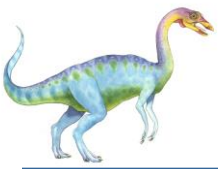


- ❑ By the time that T_2 **starts**, the **older process T_1** will have **obtained most of their locks**:



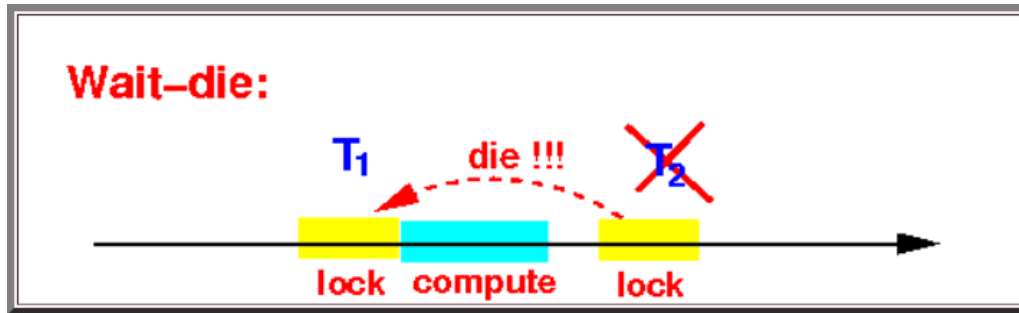
- ❑ A **younger (just starting) process T_2** will make **many lock requests !!**





Comparing

- ❑ In the **wait-die** method, a **younger** (just starting) **process T2** will **die (= abort)** if T2 requests **a lock held** by the **older process T1**:



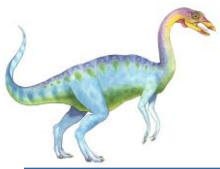
- ❑ Since **younger processes** makes **many lock requests**:

- There will be **more aborts** (of **young processes**) in the **wait-die method**

- ❑ **Comment:**

- It looks like that the wound-wait method will have better performance...





Comparing

- ❑ Comparing the **amount** of **waste work** in **aborted process** in wait-die and wound-wait
- ❑ The **wound-wait** method may have **fewer aborts**, **however**; it's **likely** that:

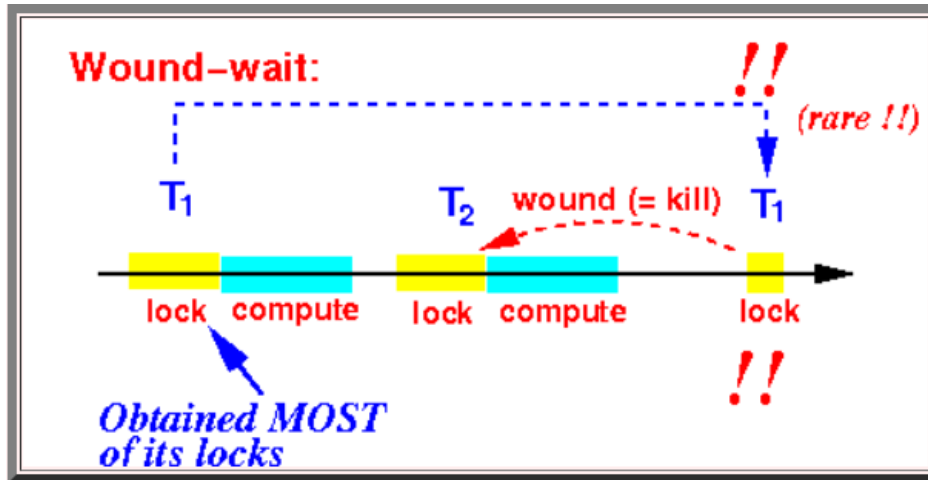
- An **aborted (younger) process** in the **wound-wait** method has **performed more work** than an **aborted (younger) process** in the **wait-die** method !!!





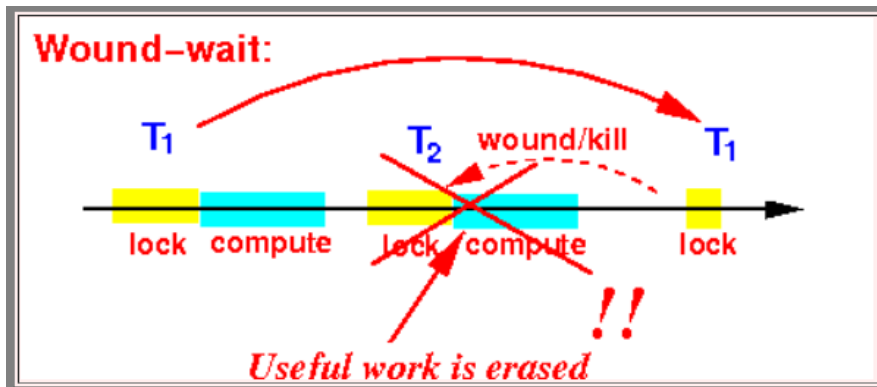
Comparing

- Consider a process in the wound-wait method that gets aborted:



Notice that the **aborted process must** hold some locks !!!

- I.e.: the **aborted process** has performed some **read/write operations** (= useful work):



The **useful work** is **erased** (rolled back) !!!

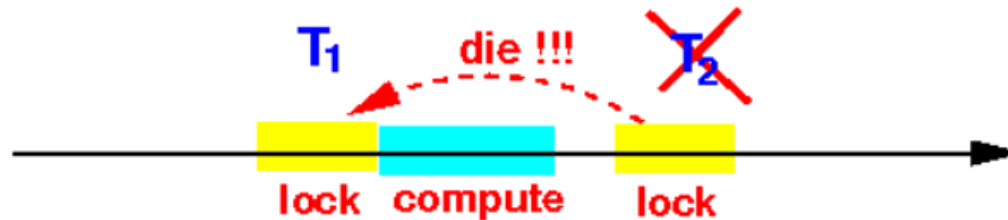




Comparing

- Consider a process in the wait-die method that gets aborted:

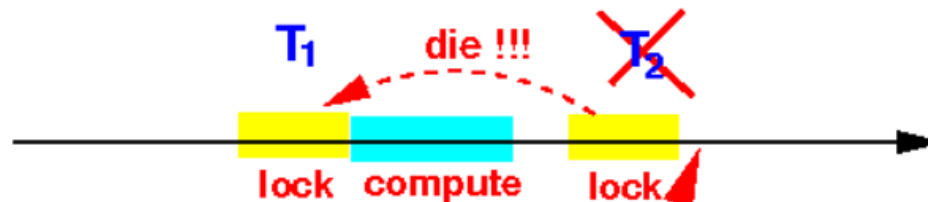
Wait-die:



Notice that the **aborted process** may **not** hold any lock at all !!!

- I.e.: the **aborted process** has **not** performed any **read/write operations** (= no useful work):

Wait-die:



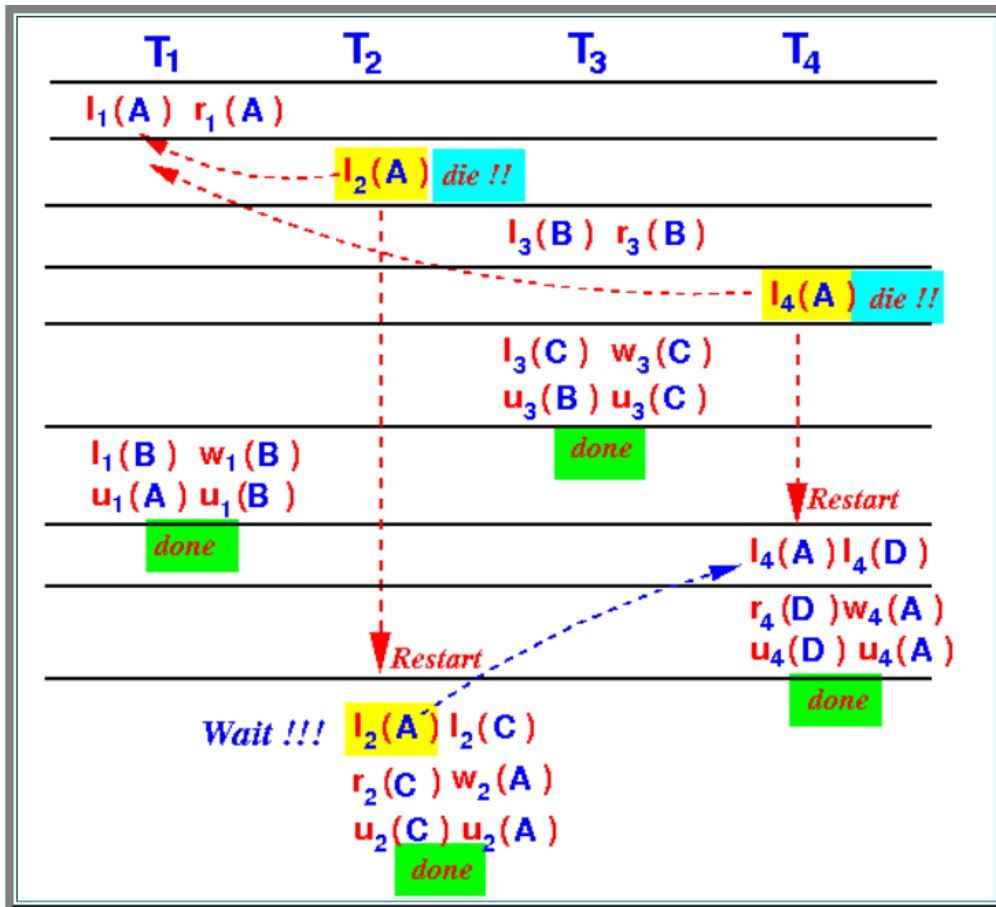
No useful work done !!!





Comparing

- ❑ Consider the previous examples:
- ❑ **Wait-die** locking method



There are 2 transaction aborts (that's 50% of the # processes !!!)

However:

Both **aborted** processes has done **no useful work** !!!

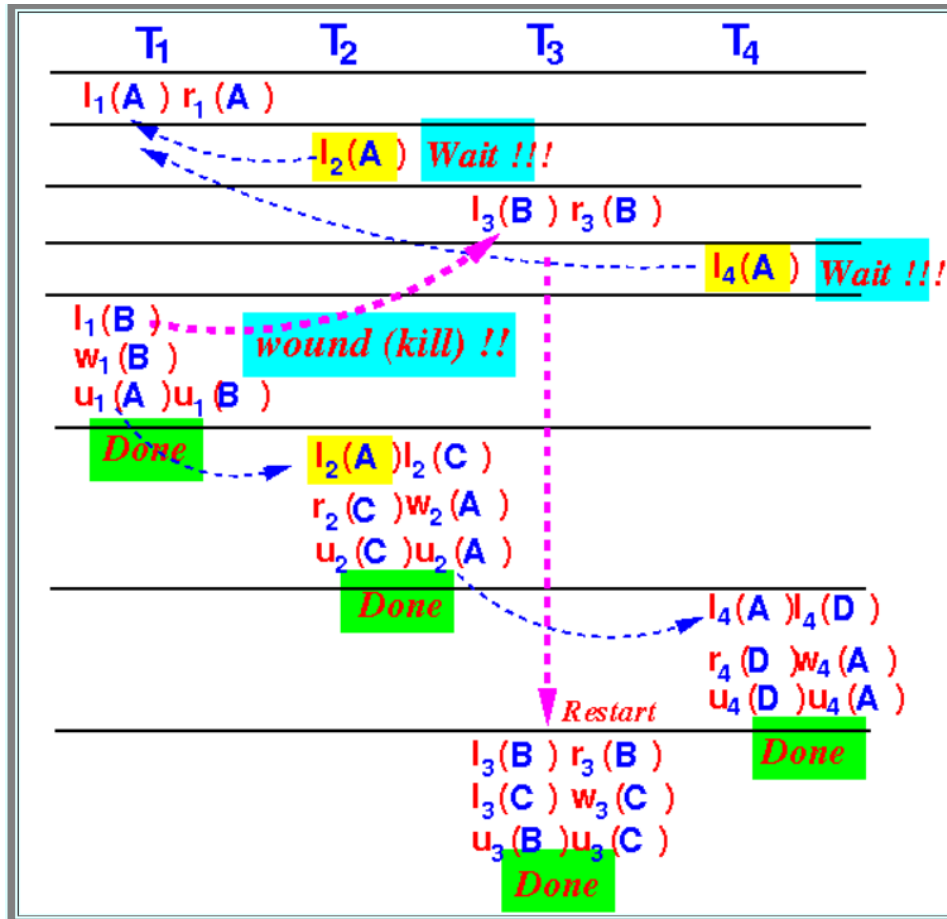
(The **aborted** processes has **not** performed any **read/write operation** !!!)





Comparing

❑ Wound-wait locking method



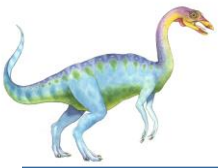
There is only **1** process aborts
(less than in wait-die !)

However:

The **aborted** processes has
done **some useful work** !!!

(Process T₃ has read B !!!)





Comparing

□ Summary

- Wait-die has more roll backs; but the processes have performed little to no work
- Wound-wait has less roll backs; but the processes that got aborted has done some work !!!

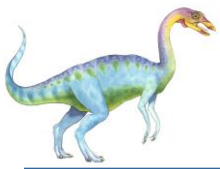




Deadlock Detection

Using **timeout** to **detect deadlock**





Deadlock detection

❑ Fact:

- When a **process T** is involved in a **deadlock**, the transaction T **will not finish**

❑ Simple way to detect if a process is a candidate to be **involved in a deadlock**:

- Set a **timer** for each operation
- When the **timer expires**, the transaction is assumed to be involved in a deadlock

❑ Note:

- This method is quite **reliable** because most operations (read/write) **does not take very long !!!**





Deadlock detection

❑ Example:

- MySQL uses **timeout** to detect deadlocks

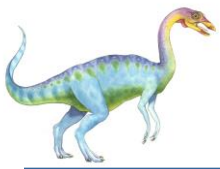
Experiment:

User 1	User 2
Don't do anything...	<pre>START TRANSACTION; UPDATE employee SET salary=salary + 0 WHERE lname='Smith'; >> 1 row updated</pre>
<pre>START TRANSACTION; UPDATE employee SET salary=salary + 0 WHERE lname='Smith'; >> Update HANGS !!! (WAIT about 1 min TIME) ERROR 1205: Lock wait timeout exceeded; try restarting transaction</pre>	Don't do anything...

- Open 2 windows and run **sqlroot** and log in as **cheung**:
- use **companyDB**
- Employee:

fname	lname	salary
John	Smith	20900.00
Frankl	Wong	50000.00
Alicia	Zelaya	25000.00
Jennif	Wallace	43000.00
Ramesh	Narayan	38000.00
Joyce	English	25000.00
Ahmad	Jabbar	25000.00
James	Borg	55000.00





Deadlock Detection

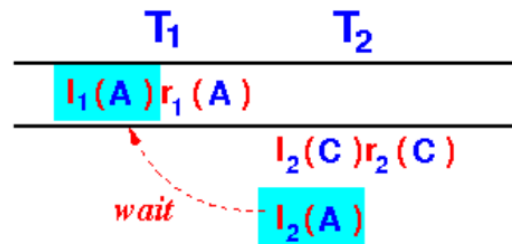
Detecting deadlocks using **wait-for graphs**





Deadlock detection

- **Wait-for graph** is a graph where:
 - **Node** represents a **process**
 - **Edge** $i \Rightarrow j$ represents: the **process** i is **waiting for a lock** held by the **process** j



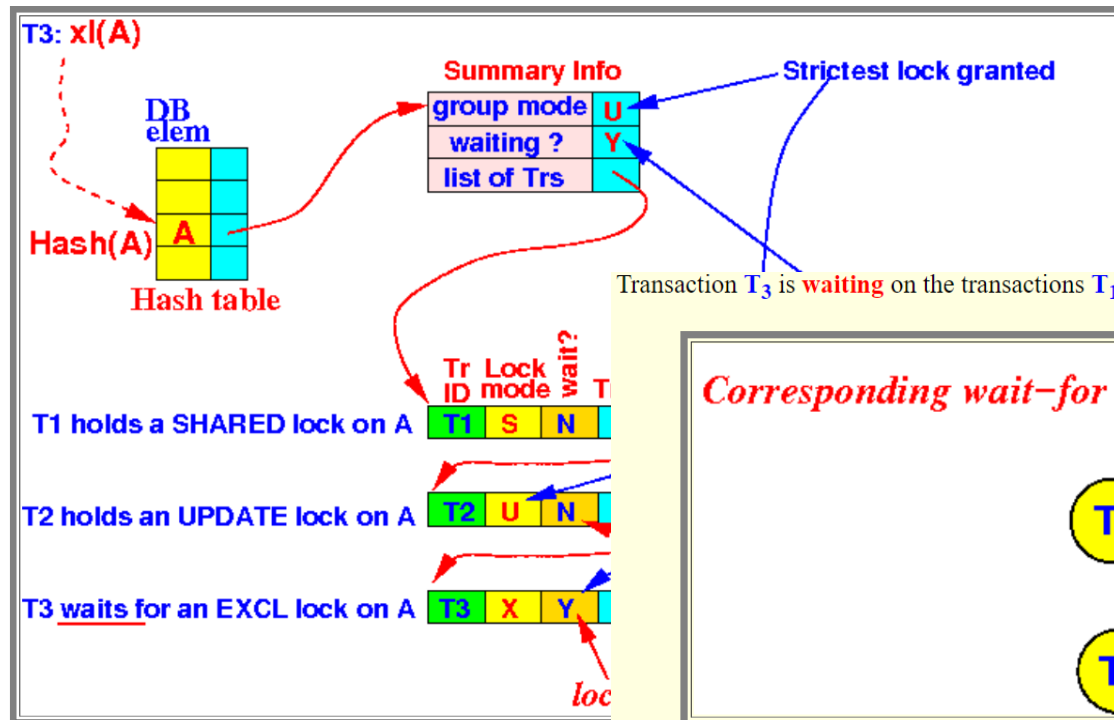
Wait-for graph:





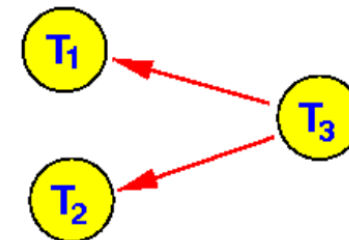
Deadlock detection

- ❑ The **wait-for** graph can be constructed using the information stored in the lock table entries



represents the fact that:

Corresponding wait-for graph:

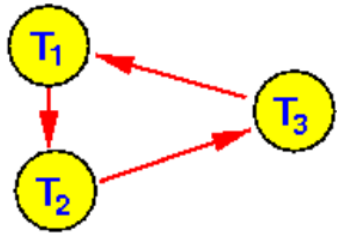




Deadlock detection

- ❑ Deadlock occurs in a circular wait situation

Circular wait:



- ❑ Each process is waiting for some process to complete
- ❑ **Result: No transaction can proceed forward**



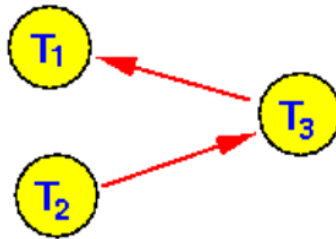


Deadlock detection

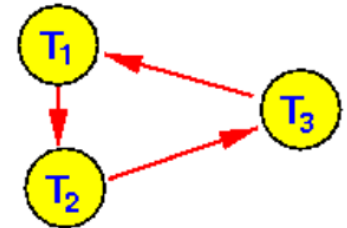
□ If the wait-for graph has no cycles, then:

- There is no deadlock

Wait-for graph with no cycle:



Circular wait:



□ Completion order:

- T₁ will complete first (and releases its locks)
- Then T₃ can obtain its locks after T₁ unlocks and complete next (and releases its locks)
- Finally, T₂ can obtain its locks after T₃ unlocks and complete next (and releases its locks)





Deadlock detection

❑ Deadlock detect in a scheduler

➤ A process scheduler that uses locking to ensure serializability must

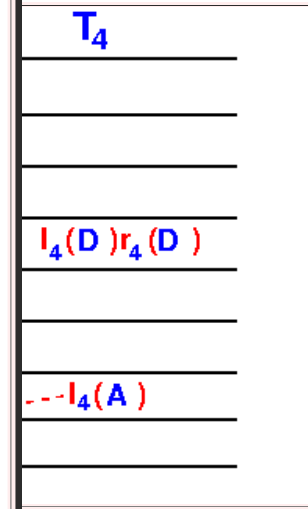
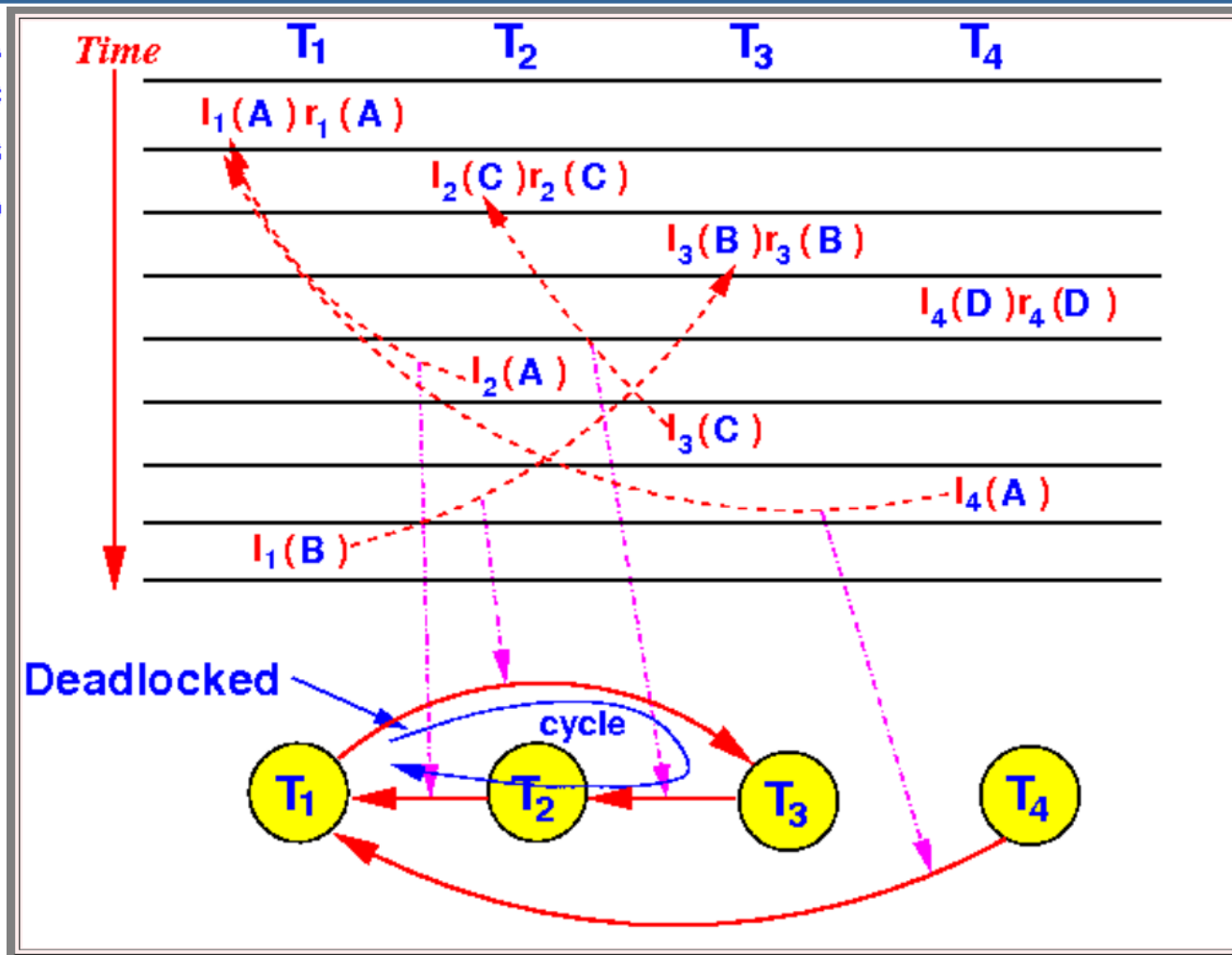
- Maintain a **wait-for** graph on all processes
- If the wait-for graph contains a cycle:
 - The scheduler will **abort** one of the processes in the cycle





Deadlock detection

$T_1: I_1(A)$
 $T_2: I_2(C)$
 $T_3: I_3(B)$
 $T_4: I_4(D)$



The process manager will now have to abort some process T_1 , T_2 and T_3 that are involved in the deadlock





Revisit: banker's algorithm

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

What are safe sequences?





Revisit: banker's algorithm

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

1. The Content of the need matrix can be calculated by using the formula given below:

$$\text{Need} = \text{Max} - \text{Allocation}$$

Process	Need		
	A	B	C
P ₀	3	2	1
P ₁	1	1	0
P ₂	5	0	1
P ₃	7	3	3
P ₄	0	0	0





Revisit: banker's algorithm

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

Process	Need		
	A	B	C
P ₀	3	2	1
P ₁	1	1	0
P ₂	5	0	1
P ₃	7	3	3
P ₄	0	0	0

For process P₀, Need = (3, 2, 1) and
Available = (2, 1, 0)
Need ≤ Available = False
So, the system will move to the next process.





Revisit: banker's algorithm

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

Process	Need		
	A	B	C
P ₀	3	2	1
P ₁	1	1	0
P ₂	5	0	1
P ₃	7	3	3
P ₄	0	0	0

For Process P1, Need = (1, 1, 0)

Available = (2, 1, 0)

Need <= Available = True

Request of P1 is granted.

Available = Available + Allocation

= (2, 1, 0) + (2, 1, 2)

= (4, 2, 2) (New Available)





Revisit: banker's algorithm

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

Process	Need		
	A	B	C
P ₀	3	2	1
P ₁	1	1	0
P ₂	5	0	1
P ₃	7	3	3
P ₄	0	0	0

For Process P2, Need = (5, 0, 1)

Available = (4, 2, 2)

Need <= Available = False

So, the system will move to the next process.





Revisit: banker's algorithm

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

Process	Need		
	A	B	C
P ₀	3	2	1
P ₁	1	1	0
P ₂	5	0	1
P ₃	7	3	3
P ₄	0	0	0

For Process P3, Need = (7, 3, 3)

Available = (4, 2, 2)

Need <= Available = False

So, the system will move to the next process.





Revisit: banker's algorithm

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

Process	Need		
	A	B	C
P ₀	3	2	1
P ₁	1	1	0
P ₂	5	0	1
P ₃	7	3	3
P ₄	0	0	0

For Process P4, Need = (0, 0, 0)

Available = (4, 2, 2)

Need <= Available = True

Request of P4 is granted.

Available = Available + Allocation

= (4, 2, 2) + (1, 1, 2)

= (5, 3, 4) now, (New Available)





Revisit: banker's algorithm

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

Process	Need		
	A	B	C
P ₀	3	2	1
P ₁	1	1	0
P ₂	5	0	1
P ₃	7	3	3
P ₄	0	0	0

Now again check for Process P2, Need = (5, 0, 1)

Available = (5, 3, 4)

Need <= Available = True

Request of P2 is granted.

Available = Available + Allocation
= (5, 3, 4) + (4, 0, 1)
= (9, 3, 5) now, (New Available)





Revisit: banker's algorithm

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

Process	Need		
	A	B	C
P ₀	3	2	1
P ₁	1	1	0
P ₂	5	0	1
P ₃	7	3	3
P ₄	0	0	0

The system allocates all the needed resources to each process. So, we can say that the system is in a safe state.

Now again check for Process P₀, = Need (3, 2, 1)

= Available (9, 5, 5)

Need <= Available = True

So, the request will be granted to P₀.

Safe sequence: < P₁, P₄, P₂, P₃, P₀>





Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?
 - How many processes will need to be rolled back?
 - ▶ one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked threads “caused” the deadlock.



End of Chapter 8

